

MULTILINGUAL SIGNBOARD TRANSLATOR USING OCR

**A project report submitted in partial fulfillment of the requirement for the
Award of the Degree of**

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

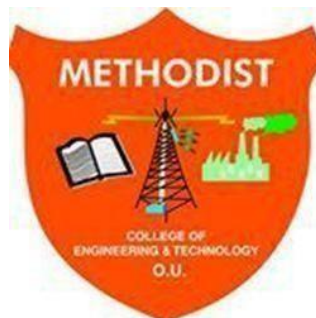
by

MOHAMMAD RIZWAN (160721747007)

KESHETTY RAMYA SRI (160721747032)

RAPARTHI BHOOMIKA (160721747058)

Under the Guidance of
Mrs. Unnati Khanapurkar,
Assistant Professor, Dept. of CSE



**Department of Artificial Intelligence and Data Science,
Methodist College of Engineering and Technology,
King Koti, Abids, Hyderabad-500001.
2024-2025**

**Methodist College of Engineering and Technology,
King Koti, Abids, Hyderabad-500001.**

Department of Artificial Intelligence and Data Science



DECLARATION BY THE CANDIDATES

We, **MOHAMMAD RIZWAN (160721747007)**, **KESHETTY RAMYA SRI (160721747032)**, and **RAPARTHI BHOOMIKA (160721747058)**, students of Methodist College of Engineering and Technology, pursuing Bachelor's degree in Artificial Intelligence and Data Science, hereby declare that this Project report entitled "**MULTILINGUAL SIGNBOARD TRANSLATOR USING OCR**", carried out under the guidance of **Mrs. Unnati Khanapurkar**, submitted in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Artificial Intelligence and Data Science. This is a record work carried out by us and the results embodied in this report have not been reproduced/copied from any source.

MOHAMMAD RIZWAN (160721747007)
KESHETTY RAMYA SRI (160721747032)
RAPARTHI BHOOMIKA (160721747058)

**Methodist College of Engineering and Technology,
King Koti, Abids, Hyderabad-500001.**

Department of Artificial Intelligence and Data Science



CERTIFICATE BY THE SUPERVISOR

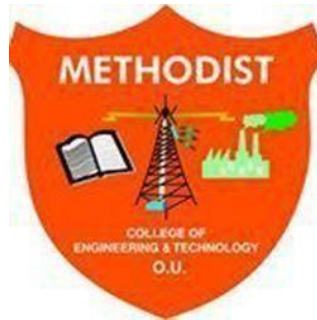
This is to certify that this Dissertation on project work entitled “**MULTILINGUAL SIGNBOARD TRANSLATOR USING OCR**” by **MOHAMMAD RIZWAN (160721747007)**, **KESHETTY RAMYA SRI (160721747032)**, and **RAPARTHI BHOOMIKA (160721747058)** submitted in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Artificial Intelligence and Data Science, during the academic year 2024-2025, is a bonafide record of work carried out by them.

Mrs. Unnati Khanapurkar,
Assistant Professor,
Dept. of CSE.

Date:

**Methodist College of Engineering and Technology,
King Koti, Abids, Hyderabad-500001.**

Department of Artificial Intelligence and Data Science



CERTIFICATE BY HEAD OF THE DEPARTMENT

This is to certify that this Dissertation on project work entitled “**MULTILINGUAL SIGNBOARD TRANSLATOR USING OCR**” by **MOHAMMAD RIZWAN (160721747007)**, **KESHETTY RAMYA SRI (160721747032)**, and **RAPARTHI BHOOMIKA (160721747058)** submitted in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Artificial Intelligence and Data Science, during the academic year 2024-2025, is a bonafide record of work carried out by them.

Dr. P. LAVANYA,
Professor &
Head of the Department.

Date:

**Methodist College of Engineering and Technology,
King Koti, Abids, Hyderabad-500001.**

Department of Artificial Intelligence and Data Science



PROJECT APPROVAL CERTIFICATE

This is to certify that this Dissertation on project work entitled “**MULTILINGUAL SIGNBOARD TRANSLATOR USING OCR**” by **MOHAMMAD RIZWAN (160721747007)**, **KESHETTY RAMYA SRI (160721747032)**, and **RAPARTHI BHOOMIKA (160721747058)** submitted in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Artificial Intelligence and Data Science, during the academic year 2024-2025, is a bonafide record of work carried out by them.

INTERNAL

EXTERNAL

HOD

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our project guide, **Mrs. Unnati Khanapurkar, Assistant Professor, CSE**, for giving us the opportunity to work on this topic. It would never be possible for us to take this project to this level without her innovative ideas and her relentless support and encouragement.

We would like to thank our project coordinator, **Dr. Shruthi SK, Associate Professor, CSE**, who helped us by being an example of high vision and pushing towards greater limits of achievement.

Our sincere thanks to **Dr. P. Lavanya, Professor and Head of the Department of Computer Science and Engineering**, for her valuable guidance and encouragement, which has played a major role in the completion of the project, and for helping us by being an example of high vision and pushing towards greater limits of achievement.

We would like to express a deep sense of gratitude towards **Dr. Prabhu G. Benakop, Principal, Methodist College of Engineering and Technology**, for always being an inspiration and for always encouraging us in every possible way.

We would like to express a deep sense of gratitude towards **Dr. Lakshmipathi Rao, Director, Methodist College of Engineering and Technology**, for always being an inspiration and for always encouraging us in every possible way.

We are indebted to the Department of Artificial Intelligence and Data Science and Methodist College of Engineering and Technology for providing us with all the required facility to carry out our work in a congenial environment. We extend our gratitude to the AI&DS Department staff for providing us with the needful time to time whenever requested.

We would like to thank our parents for allowing us to realize our potential, all the support they have provided us over the years was the greatest gift anyone has ever given us, and also for teaching us the value of hard work and education. Our parents have offered us with tremendous support and encouragement, thanks to our parents for all the moral support and the amazing opportunities they have given us over the years.

Department of Artificial Intelligence and Data Science

Vision & Mission

VISION

To become a leader in providing Artificial Intelligence and Data Science education with emphasis on knowledge and innovation.

MISSION

- M1:** To offer flexible programs of study with collaborations to suit industry needs.
- M2:** To provide quality education and training through novel pedagogical practices.
- M3:** To expedite high performance of excellence in teaching, research, and innovations.
- M4:** To impart moral, ethical valued education with social responsibility.

Program Educational Objectives

Graduates of Artificial Intelligence and Data Science at Methodist College of Engineering and Technology will be able to:

- PEO1:** Apply technical concepts, analyze, and synthesize data to Design and create novel products and solutions for real-life problems.
- PEO2:** Apply the knowledge of Artificial Intelligence and Data Science to pursue higher education with due consideration to the environment and society.
- PEO3:** Promote collaborative learning and the spirit of teamwork through multidisciplinary projects.
- PEO4:** Engage in lifelong learning and develop entrepreneurial skills.

Program Specific Outcomes

At the end of 4 years, Artificial Intelligence and Data Science graduates at MCET will be able to:

- PSO1:** Apply the knowledge of Artificial Intelligence and Data Science in various domains like networking and data mining to manage projects in multidisciplinary environments.
- PSO2:** Develop software applications with open-ended programming environments.
- PSO3:** Design and develop solutions by following standard software engineering principles and implement by using suitable programming languages and platforms.

Program Outcomes (POs)

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

ABSTRACT

In a multilingual world, navigating signboards written in unfamiliar languages poses significant challenges, especially for travelers and immigrants. To address this issue, this project introduces a solution titled "Multilingual Signboard Translator Using OCR", which integrates Optical Character Recognition (OCR), language translation, and speech synthesis into a unified system. The application allows users to either upload an image of a signboard or provide live audio input. Using Tesseract OCR, the system extracts text from images, and the live audio is converted into text using a speech recognizer. The extracted content is then translated into a user-selected language using the Google Translate API, and finally, the translated text is converted into speech via Google Text-to-Speech (gTTS). The application is built with the Flask web framework and features an intuitive interface, robust input clarity checks, and error-handling mechanisms to ensure a smooth user experience. This system serves as a practical tool for breaking language barriers in public spaces and can be particularly useful in fields like tourism, international transportation, and emergency services. The application has been tested for accuracy in OCR detection, translation quality, and responsiveness across different languages and formats, which provides us with more than 90% accuracy, and in case of a clearer image, it can be above 95%. Results indicate high reliability in extracting and translating text, even from images of varying clarity. By combining multiple technologies into a single, accessible web application, this project contributes to the growing need for inclusive communication tools in a globalized environment. Future work may include support for handwriting recognition, offline usage, and additional language dialects for wider applicability.

The outcome of this project is a functioning system that can accurately recognize and translate multilingual text from images of signboards, live audio in real time, and convert it into audio of the user's preferred language. This system helps overcome language barriers in public spaces by combining OCR (Optical Character Recognition) with translation technologies. The application helps to strengthen skills in AI/ML, OCR, and app development—applicable for the Ministry of Tourism, Government of India, travelling apps, freelance, or entrepreneurial paths. The application can be used prospectively for the. Real-time Images of signboards were collected across India. The SDG Indicators addressed by project are 4.4.1, 4.7.1, 9.5.1,9.5.2, 9.c.1,10.2.1, 10.3.1,11.2.1, 11.7.1, 11.7.2

TABLE OF CONTENTS

1. INTRODUCTION.....	1
1.1. BACKGROUND	1
1.2. MOTIVATION	1
1.3. PROBLEM STATEMENT	2
1.4. OBJECTIVES	2
1.5. SCOPE	3
2. LITERATURE REVIEW	4
2.1. EXISTING SYSTEMS AND TECHNIQUES	4
2.1.1. OCR-Based Translation Systems.....	4
2.1.2. Language Translation Systems	4
2.1.3. Text-to-Speech (TTS) Systems.....	5
2.1.4. Multimodal Translation Systems	5
2.2. LITERATURE SURVEY	6
2.3. RESEARCH GAPS	10
3. SYSTEM DESIGN.....	11
3.1. OVERVIEW OF THE SYSTEM ARCHITECTURE	11
3.1.1. System Components.....	11
3.1.2. System Workflow	12
3.1.3. Dataflow Diagrams	13
3.1.4. UML Diagram.....	14
3.1.5. Sequence Diagram	15
3.1.6. Activity Diagram	15
3.2. TECHNOLOGIES USED.....	16
3.2.1. Python	16
3.2.2. Flask.....	16
3.2.3. OCR & Tesseract OCR.....	17
3.2.4. OpenCV	18
3.2.5. Speech Recognition	18
3.2.6. Google Translate API	19
3.2.7. Google Text-to-Speech API.....	19
3.2.8. HTML/CSS/JavaScript	20
3.3. SYSTEM CONSTRAINTS	21
3.4. HARDWARE AND SOFTWARE REQUIREMENTS	21

3.4.1.	Hardware Requirements.....	21
3.4.2.	Software Requirements.....	21
3.4.3.	Google APIs.....	21
4.	MODULES AND IMPLEMENTATION	22
4.1.	IMAGE INPUT MODULE.....	22
4.2.	AUDIO INPUT MODULE.....	23
4.3.	OCR TEXT EXTRACTION.....	23
4.4.	LANGUAGE DETECTION AND TRANSLATION	24
4.5.	TEXT-TO-SPEECH CONVERSION.....	25
4.6.	FRONTEND DESIGN AND USER INTERACTION.....	25
4.7.	FLASK BACKEND INTEGRATION	26
5.	TESTING AND EVALUATION	27
5.1.	TEST STRATEGY	27
5.2.	SAMPLE TEST CASES.....	27
5.3.	LOG OUTPUTS	28
6.	SCREENSHOTS.....	29
6.1.	WEB INTERFACE.....	29
6.2.	IMAGE UPLOADS	30
6.2.1.	English Signboard Upload	30
6.2.2.	Hindi Signboard Upload	32
6.3.	SPEAK LIVE.....	34
7.	RESULTS AND ANALYSIS	35
7.1.	ACCURACY OF OCR DETECTION.....	35
7.2.	TRANSLATION QUALITY.....	36
7.3.	CLARITY VERIFICATION	36
7.4.	USABILITY AND USER FEEDBACK	36
8.	CONCLUSION	37
9.	FUTURE ENHANCEMENTS.....	38
10.	REFERENCES.....	39
	APPENDIX.....	41
A.	SAMPLE CODE.....	41
B.	ERROR LOGS AND RESOLUTIONS.....	47
	QUALITY ANALYSIS AND ALIGNMENT OF STUDENT PROJECTS WITH PROGRAM OUTCOMES (POs) AND SUSTAINABLE DEVELOPMENT GOALS (SDGs).....	48
	RESOURCES.....	52

LIST OF FIGURES

Fig 3.1.2: Architecture.....	12
Fig 3.1.3: Dataflow Diagram Level-0.....	13
Fig 3.1.3.1: Dataflow Diagram Level-1.....	14
Fig 3.1.4: Use Case Diagram.....	14
Fig 3.1.5: Sequence Diagram.....	15
Fig 3.1.6: Activity Diagram.....	15
Fig 6.1: Web Interface.....	29
Fig 6.2.1: Signboard 1.....	30
Fig 6.2.1.1: Translation of English Signboard ("Welcome") to Hindi.....	31
Fig 6.2.2: Signboard 2.....	32
Fig 6.2.2.1: Translation of Hindi Signboard to Tamil.....	33
Fig 6.3: Translation of the Live Audio Input.....	34
Fig 7.1: Test Case-1.....	35
Fig 7.1.1: Test Case-2.....	35
Fig 7.1.2: Test Case-3.....	35

LIST OF TABLES

Table 2.2: Literature Survey.....	9
Table 5.2: Test Cases.....	28
Table B.1: Error Logs and Fixes.....	47

1. INTRODUCTION

1.1. BACKGROUND

Globalization has brought together people from diverse linguistic backgrounds in today's interconnected world. With the increasing movement of people across states and countries for tourism, education, work, or migration, the challenge of language barriers has become more prominent than ever. Signboards, which are crucial in conveying vital information in public places such as streets, airports, railway stations, hospitals, and historical monuments, often present information in the local language. While this is effective for native speakers, non-native speakers or international tourists may face difficulty understanding the displayed content.

Language barriers in such scenarios can lead to confusion, misdirection, or even serious consequences, especially in emergency or healthcare settings. Therefore, developing systems that can translate signboards in real time and provide audio feedback can significantly aid in information dissemination and public convenience.

With the advancements in Optical Character Recognition (OCR), Natural Language Processing (NLP), and Text-to-Speech (TTS) technologies, it is now feasible to design intelligent systems capable of reading text from images, translating it to a preferred language, and converting it into speech. These technologies offer the potential to create multilingual assistants that bridge language gaps in practical and meaningful ways.

1.2. MOTIVATION

The primary motivation for developing the “Multilingual Signboard Translator Using OCR” is the need to facilitate cross-lingual communication in real-world settings. While travelling in regions where the local language is unfamiliar, users often struggle to understand signs, especially if there is no English or common language translation provided. This limits accessibility and can result in poor user experiences.

Furthermore, the current availability of translation apps often requires manual input, does not verify the clarity of the image, or may not handle both textual and audio inputs simultaneously. There exists a need for a more seamless, accessible, and inclusive application that not only reads text from signboards but also speaks the translation out loud, and also takes voice inputs, making it beneficial for both literate and illiterate users, visually impaired individuals, and those with reading difficulties.

Statistical Impact:

- 80% of travelers face language barriers in foreign countries.
- 91% of visually impaired individuals rely on audio tools for navigation.
- The language services market is expected to reach \$57 billion by 2030.

In today's interconnected world, overcoming language barriers is essential, especially in public spaces where signboards convey critical information. This project addresses challenges such as:

- ❖ **Global Multilingualism:** Over 7,100 languages exist worldwide, with India alone housing 22 official languages and 19,500+ dialects. For travelers, understanding signboards in unfamiliar languages leads to confusion.
- ❖ **Tourism Growth:** The global tourism market, projected to reach \$1.8 trillion by 2030, highlights the need for multilingual tools, with India witnessing an 11% annual rise in international tourist arrivals (2023).
- ❖ **Accessibility for Visually Impaired:** Over 253 million visually impaired individuals globally can benefit from audio-based solutions for signboards.
- ❖ **Advancements in AI:** Technologies like OCR, translation, and text-to-speech have made real-time multilingual solutions feasible, scalable, and accessible.

1.3. PROBLEM STATEMENT

The research addresses the challenge of understanding signboards in unfamiliar languages by developing a system that extracts text from images or live audio, translates it into the user's chosen language, and converts it to speech. The primary question is how AI can be utilized to develop a multilingual, accessible text-to-speech system for a diverse range of users. The innovative contribution is a single, real-time solution combining OCR, translation, and speech synthesis for practical use. This study is significant since it bridges language barriers and enhances inclusivity while utilizing reliable AI frameworks to ensure scalability and impact.

1.4. OBJECTIVES

The primary objective of this project is to design and implement a web-based application that performs real-time multilingual signboard translation using OCR. The detailed objectives include:

- To develop an OCR engine integrated with multiple language support for extracting text from uploaded signboard images.
- To implement a live audio input module that allows users to speak instructions or queries for translation.
- To incorporate a language translation module that accurately converts the extracted text to the language of the user's choice.
- To build a Text-to-Speech system to generate audio of the translated content.
- To ensure the application performs image clarity checks and notifies the user when the image is not readable.
- To design a user-friendly frontend that allows easy uploading of images, speaking input, selecting languages, and receiving both textual and audio output.

1.5. SCOPE

The scope of the “Multilingual Signboard Translator Using OCR” project includes the design, development, and evaluation of a prototype web application that combines image processing, speech recognition, translation services, and speech synthesis. This project will support commonly spoken Indian languages, including English, Hindi, Tamil, Telugu, Malayalam, Kannada, Marathi, and Bengali. The OCR component will be tailored to recognize text in these languages using the Tesseract OCR engine.

The system will be designed for web deployment using Flask, allowing users to interact with it via a web browser on desktops or mobile devices. It will support two types of input: image upload (for reading signboards) and live speech input (as an alternative way to query or translate). The translation and TTS components will leverage APIs from Google Translate and Google Text-to-Speech, respectively. The application is intended for real-world use cases such as tourism, public transport, education, and signage accessibility.

2. LITERATURE REVIEW

2.1. EXISTING SYSTEMS AND TECHNIQUES

The domain of multilingual signboard translation using OCR is an intersection of multiple technological areas, including Optical Character Recognition (OCR), language translation, and Text-to-Speech (TTS) systems. Several approaches have been explored for each of these components in isolation or combination, but there are limited systems that effectively integrate all the functionalities required for a complete signboard translation system.

2.1.1. OCR-Based Translation Systems

OCR technology has been widely used for text extraction from images. Tesseract, one of the most prominent open-source OCR engines, is frequently used for recognizing text in various languages. OCR tools work by analyzing the visual structure of text in images and converting it into machine-readable data. Tesseract has been successfully used in numerous applications such as document digitization, license plate recognition, and signboard text extraction.

Research has shown that OCR accuracy heavily depends on factors such as the quality of the image, the font used, and the complexity of the text layout. In the context of multilingual text, OCR tools are limited in their ability to handle multiple scripts simultaneously, particularly when the text is not well-aligned or is obscured. Several advancements have been made to enhance the recognition capabilities of Tesseract for languages with complex scripts (e.g., Hindi, Tamil, Bengali), but accuracy can still be a challenge for noisy or unclear images.

2.1.2. Language Translation Systems

The field of machine translation (MT) has experienced significant breakthroughs in recent years. Google Translate is one of the most widely used language translation tools globally, offering support for over 100 languages, including many regional Indian languages. Google Translate is based on Neural Machine Translation (NMT) models, which have vastly improved the quality of translations by taking context into account rather than translating word by word. This has translated diverse languages more accurately, though issues such as idiomatic expressions, cultural nuances, and regional dialects remain significant challenges.

While Google Translate is highly accessible and effective, there are still limitations in handling highly specialized terminology or complex sentence structures. Moreover, real-time translation for signboards may require optimization for speed, which is not always achieved with offline models. In many cases, the text detected by OCR must be cleaned and pre-processed before it can be effectively translated.

2.1.3. Text-to-Speech (TTS) Systems

Text-to-Speech technology converts written text into spoken words. Google's gTTS (Google Text-to-Speech) API is one of the most widely used solutions for this task due to its easy integration, support for multiple languages, and high-quality voice output. In recent years, TTS systems have evolved to produce more natural-sounding voices, with advances in deep learning models for speech synthesis.

However, challenges remain in making TTS systems sensitive to different accents, pronunciations, and emotional tone. Also, language-specific quirks such as intonation, phonetics, and syllable stress pose challenges when converting text to speech, especially for languages with significant variation in pronunciation.

2.1.4. Multimodal Translation Systems

A few existing systems attempt to integrate OCR, translation, and TTS functionalities. For instance, some mobile applications allow users to point their smartphone camera at a sign, extract the text, translate it, and read it aloud. These systems generally utilize OCR libraries like Tesseract and combine them with translation APIs like Google Translate and TTS APIs like Google gTTS or Amazon Polly.

However, these applications often lack advanced error handling (such as image clarity validation), struggle with multi-language support in a single image, and are not always efficient in real-time applications. In many cases, these systems are focused on image translation alone and do not support live speech recognition or provide both text and audio outputs together.

2.2. LITERATURE SURVEY

S. No	Paper Title	Authors	Methodology	Datasets	Key Findings	Future Scope	Evaluation Metrics
1	A Robust Intelligent System for Text-Based Traffic Signs Detection and Recognition in Challenging Weather Conditions (2024)	Khalid, Sara, et al.	Utilized deep learning models (Hybrid CNN-SVM) with image preprocessing and feature extraction techniques for detection in adverse weather.	Self-collected under varying weather (rain, fog, night)	Achieved robust detection despite challenging environments.	Incorporate multi-language translation and real-time video processing.	Precision, Recall, F1-Score, Accuracy ~94.50%
2	SignboardText : Text Detection and Recognition in In-the-Wild Signboard Images (2024)	Do, Tien, et al.	Hybrid CNN-RNN model with transformer-based OCR; training using scene-specific enhancements.	SignboardText dataset (proposed by authors)	Improved accuracy on multilingual and cluttered signboards.	Integration with mobile translation apps.	F1 Score, CER, WER, BLEU score, Accuracy ~92.70%
3	Signboard detection and text recognition using artificial neural networks (2019)	Panhwar, Muhammad A., et al.	ANN with feature-based segmentation and OCR for signboard extraction.	Images from public traffic datasets	Demonstrated feasibility of simple ANNs for low-resource deployments.	Use of deeper networks or transfer learning for better performance.	Processing Time, Accuracy ~87%
4	Scene text extraction using stroke width transform for tourist translator on android platform (2016)	Chavre, Pooja, and Archana Ghotkar	SWT algorithm with connected components analysis for text localization and translation.	Tourist photos captured via Android camera	Enabled real-time on-device OCR translation.	Support for more complex scripts and cloud-based translation.	Processing Speed, Text Detection Rate, Accuracy ~85%

S. No	Paper Title	Authors	Methodology	Datasets	Key Findings	Future Scope	Evaluation Metrics
5	Automatic detection and recognition of text in traffic sign boards based on word recognizer (2016)	Xavier, Seles, and R. Reshmi	Word recognizer-based OCR with character segmentation for traffic signboards.	Custom dataset from roadside boards	Effective for isolated signs in clear daylight.	Incorporate voice synthesis and multi-language options.	Detection Rate, Character Error Rate, Accuracy ~88%
6	The image of China as a destination for tourist: translation strategies of culture-specific items in the Chinese-English food menus (2023)	Amenador, Kate Benedicta, and Zhiwei Wang	Manual analysis of translation strategies for culturally specific items in food menus.	Real-world Chinese-English menus	Cultural adaptation plays a key role in translation efficacy.	Automate cultural nuance recognition using AI.	NA
7	Traffic Signboard Recognition and Text Translation System using Word Spotting and Machine Learning (2022)	Usha, S. M., D. Mahesh Kumar, and H. B. Mahesh	Word spotting, image preprocessing, and SVM-based recognition combined with translation.	Custom Indian traffic sign dataset	Demonstrated pipeline system from detection to translation.	Enhance system for real-time mobile implementation.	Word Accuracy Rate, Latency, Accuracy ~93.10%
8	Show Me the World in My Language: Establishing the First Baseline for Scene-Text to Scene-Text Translation (2023)	Vaidya, Shreyas, et al.	Scene-text translation pipeline using transformers for end-to-end multilingual signboard translation.	SynthText, Open Images, ST2ST	Created baseline model for multilingual signboard translation.	Expand to include contextual translation and localization.	BLEU Score, Accuracy ~89.40%

S. No	Paper Title	Authors	Methodology	Datasets	Key Findings	Future Scope	Evaluation Metrics
9	Instant Arabic translation system for signboard images based on printed character recognition (2013)	Al-Hashemi, Rafeeq Abdul Rahman A., and Shoroq Almamoun Alsharari	Printed OCR system with Arabic-English translation capability.	Collected Arabic signboard images	Showed potential for instant visual translation of Arabic signboards.	Add voice assistance and real-time streaming.	Translation Time, OCR Accuracy ~85%
10	Android-based Text Reader for Partial Vision Impairment (2018)	Dhar, Rituparna, and Saswati Mukherjee	Android app with Tesseract OCR and voice output for the visually impaired.	Image samples from public signboards	Enabled accessibility for partially blind users.	Add real-time camera scanning and language options.	Readability, Response Time, Accuracy ~86%
11	Diffusionstr: Diffusion model for scene text recognition (2023)	Fujitake, Masato	Diffusion-based generative model for robust scene text recognition.	ICDAR2013, IIIT5K	Achieved superior results on complex scene-text images.	Extend to translation + speech generation.	CER, WER, Accuracy ~95%
12	Signboard Text Translator: An Essential Guide for Tourists (2023)	Ponnapalli, Srinivasu Badugu, Swetha, Sharia Zainab, and Rida Fatima	OCR + Google Translate API in an Android app pipeline.	Custom tourist sign images	Demonstrated useful mobile application for travelers.	Expand to offline translation and regional scripts.	Usability Testing, Accuracy ~88%
13	A Translator for Indian Sign Boards to English using Tesseract and SEQ2SEQ Model (2021)	Pandiyan, S. Sundara, and Charles Kelvin	Tesseract OCR + Seq2Seq model for text translation from regional languages.	Indian regional signboards	Effective for low-resource Indian languages.	Integrate speech and multiple input modalities.	BLEU Score, Translation Accuracy: 87–90%

S. No	Paper Title	Authors	Methodology	Datasets	Key Findings	Future Scope	Evaluation Metrics
14	Improved Real time Signboard Detection and Translation Using FRCNN (2021)	Dona Lal, Jothika Mohan, Renjisha T S, Saviya Davis, Divya Mohan	FRCNN for detection, OCR for recognition, Google Translate for translation.	Mixed dataset with multilingual signboards	Real-time translation achieved with high accuracy.	Add image clarity enhancement techniques.	FPS, BLEU, OCR Accuracy ~90%
15	Image to Multilingual Text Conversion for Literacy Education (2018)	Ajmal, Muhammad, et al.	Deep OCR with language detection and translation for literacy tools.	Custom literacy datasets	Useful for educational contexts with multilingual signage.	Add interactive speech generation.	Translation Accuracy ~85%, Processing Time
16	Signboard text translator: a guide to tourist (2017)	Kavati, Ilaiah, et al.	Android app integrating Tesseract OCR and text-to-speech.	Indian tourist place signboards	Designed tourist-friendly sign translation tool.	Improve translation accuracy and real-time processing.	OCR Accuracy: 83%, TTS Clarity
17	Detecting text-based image with optical character recognition for English translation and speech using Android (2015)	Ramiah, Sathiapriya, Tan Yu Liong, and Manoj Jayabalan	Image pre-processing, OCR, English translation, and text-to-speech for mobile devices.	Field-tested road signboards	Enabled OCR+Speech output for on-the-go translation.	Better UI and multilingual support.	Speed, Accuracy ~82%, Usability
18	Automatic detection and recognition of signs from natural scenes (2004)	Chen, Xilin, et al.	Early-stage image-based sign detection using connected component analysis and OCR.	Natural scene datasets	Pioneered sign detection using OCR in real-world scenes.	Improve with deep learning and real-time translation.	Detection Rate, OCR Accuracy ~80%

Table 2.2: Literature Survey

2.3. RESEARCH GAPS

Despite the progress in signboard translation systems, several significant gaps remain that warrant further exploration:

1. **Lack of Unified Pipeline:** Many systems excel in either detection, recognition, or translation, but do not provide an efficient end-to-end solution integrating all three seamlessly, especially with audio output support.
2. **Multilingual and Multimodal Support:** Existing systems often support a limited number of languages and lack adaptability to regional dialects or rare scripts. Few offer both image and audio input modes effectively.
3. **Performance in Adverse Conditions:** Many approaches struggle under poor lighting, occlusions, low-resolution images, or challenging weather, although some (like Khalid et al., 2024) have attempted to address this.
4. **Real-Time Processing Constraints:** While deep learning improves accuracy, the models tend to be computationally expensive and less suitable for real-time use on mobile or edge devices without optimization.
5. **Dataset Limitations:** There is a lack of large-scale, publicly available multilingual signboard datasets that include diverse environmental conditions and cultural contexts. Some datasets (e.g., SignboardText, 2024) are a step forward, but broader benchmarks are needed.
6. **Contextual Translation and Cultural Sensitivity:** Translating signboard text accurately while preserving cultural meaning, especially in tourism-related content (e.g., Amenador & Wang, 2023), remains a challenge.

3. SYSTEM DESIGN

3.1. OVERVIEW OF THE SYSTEM ARCHITECTURE

The system for multilingual signboard translation, utilizing OCR, translation, and text-to-speech (TTS), has been designed to provide a seamless and user-friendly experience. The core objective is to create a Flask web application that allows users to upload images of signboards, detect text through Optical Character Recognition (OCR), translate the detected text into a specified language, and then generate the translated text as audio output. The system architecture includes the integration of three major components:

- ❖ **OCR (Tesseract):** Extract text from signboard images.
- ❖ **Speech Recognition:** The audio input is recognized by the speech recognizer.
- ❖ **Translation (Google Translate API):** To translate the extracted text from both inputs into a user-selected language.
- ❖ **Text-to-Speech (gTTS):** To convert the translated text into speech, enabling users to hear the translation.

The system employs a client-server architecture, with the Flask web application serving as the server to handle user requests, process data, and return results. The client interacts with the system through a user-friendly web interface. Below is a detailed breakdown of the system architecture.

3.1.1. System Components

1. **Frontend (User Interface):** The frontend consists of HTML, CSS, and JavaScript files, which are housed within the templates/ and static/ directories of the Flask app. The user can upload an image or provide live audio input, and the interface will display feedback on the progress of the translation and allow the user to listen to the generated audio.
2. **Backend (Flask Application):** The backend is built using the Flask web framework, which handles incoming HTTP requests from the user. It processes the image (if uploaded), runs the OCR on it, handles the translation request, and passes the text to the TTS system. Flask then returns the translated audio file to the user.
3. **OCR Module:** The Tesseract OCR engine is used for recognizing and extracting text from the uploaded images. The OCR module takes in an image, processes it, and outputs the recognized text. It provides an accuracy of more than 90%.
4. **Speech Recognition Module:** The speech recognizer extracts the voice captured by the microphone when a user speaks and converts it into text.
5. **Translation Module:** The Google Translate API is used to translate the recognized text into the selected language. The system can handle multiple languages, making it versatile for users with different language preferences.

6. **TTS Module:** The gTTS (Google Text-to-Speech) API is used to convert the translated text into spoken words. This allows users to listen to the translated text in their preferred language.
7. **Error Handling:** The system includes error handling to ensure smooth user interaction. If the image quality is low or if no text is detected, the system will prompt the user to upload a clearer image or provide a different input. Additionally, the system supports feedback messages for errors in the translation or TTS processes.

3.1.2. System Workflow

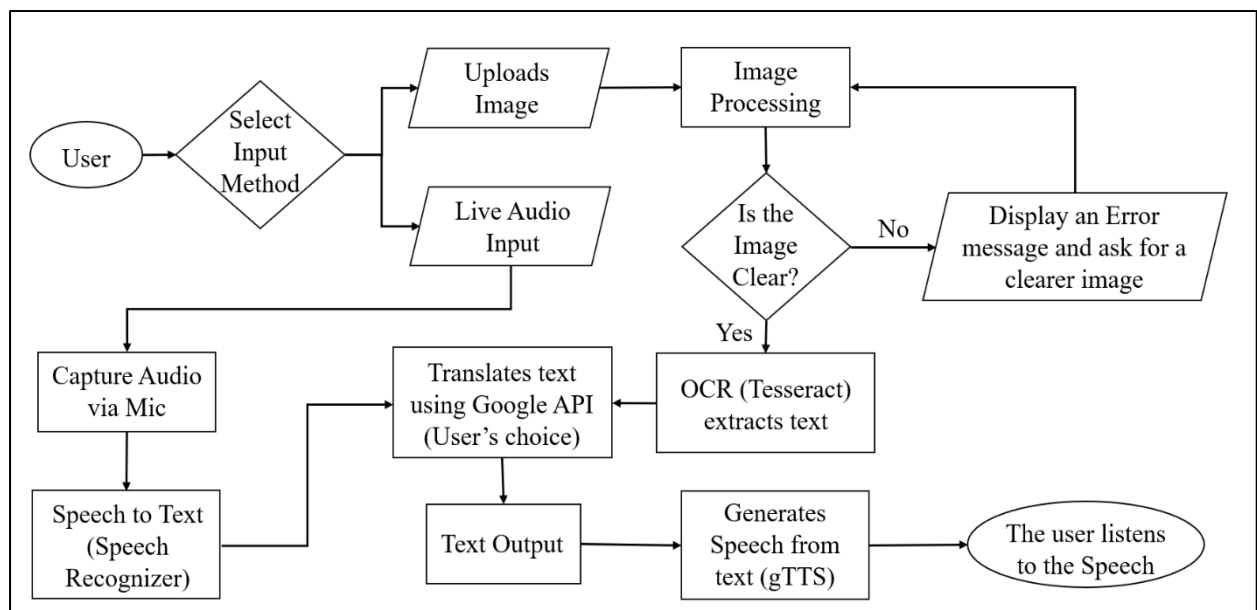


Fig 3.1.2: Architecture

The system operates in the following sequence of steps:

1. User Input:

- The user interacts with the frontend interface to either upload an image or provide live audio input.
- In the case of image input, the image is sent to the backend Flask application for processing.

2. Image Processing (OCR):

- The uploaded image is passed to the OCR module (Tesseract) for text extraction.
- If text is detected, the extracted text is sent to the translation module for further processing.

3. Audio Input:

- The audio is captured by the mic and is passed to the speech recognizer.
- The speech recognizer extracts the speech when the user speaks and converts it into text.

4. Text Translation:

- The extracted text is sent to the Google Translate API, which returns the translated text in the user's chosen language.
- The translation module handles the request and ensures that the translated text is contextually accurate.

5. Text-to-Speech Conversion:

- The translated text is passed to the gTTS module, which generates an audio file of the translated text.
- The audio file is then stored in the static/audio/ folder, ready to be accessed by the user.

6. User Feedback:

- The translated text is displayed on the web interface, and the user can listen to the translation by clicking the "Play" button.
- If the uploaded image is not very clear, then the system sends a message to the user to upload a clearer one.

7. Error Handling:

- If OCR fails to detect any text or if the image is unclear, the system will notify the user.
- If the translation service encounters an issue, an error message will be shown, prompting the user to try again or check their input.

3.1.3. Dataflow Diagrams

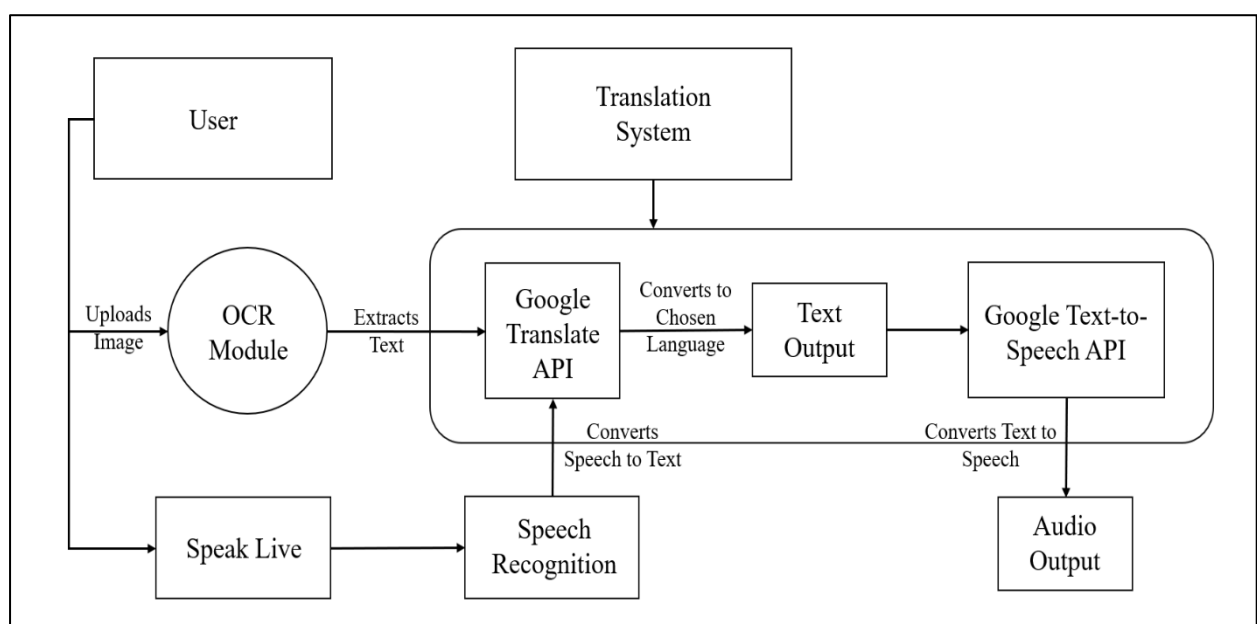


Fig 3.1.3: Dataflow Diagram Level-0

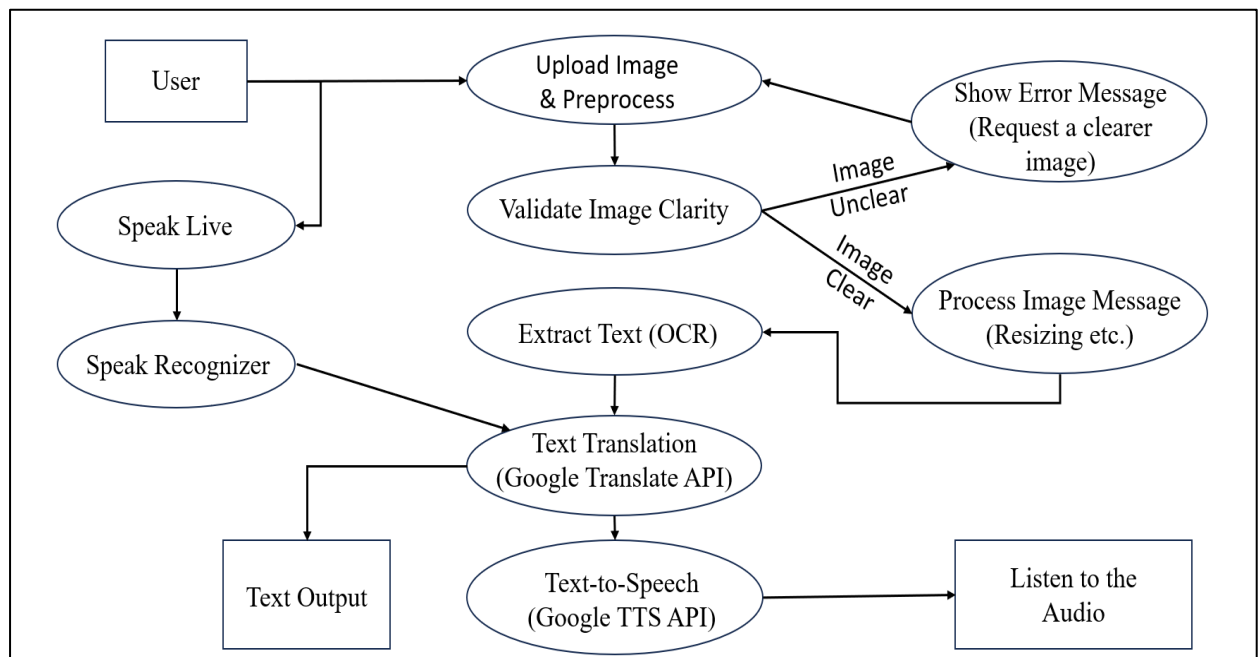


Fig 3.1.3.1: Dataflow Diagram Level-1

3.1.4. UML Diagram

Use Case Diagram:

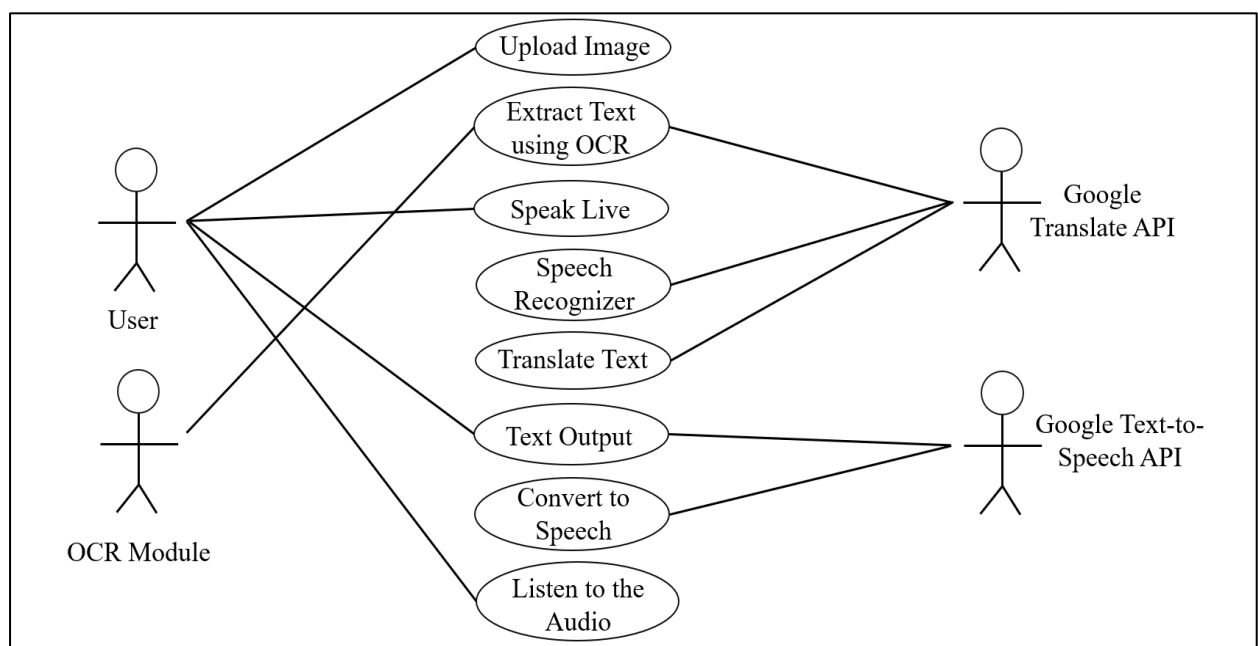


Fig 3.1.4: Use Case Diagram

3.1.5. Sequence Diagram

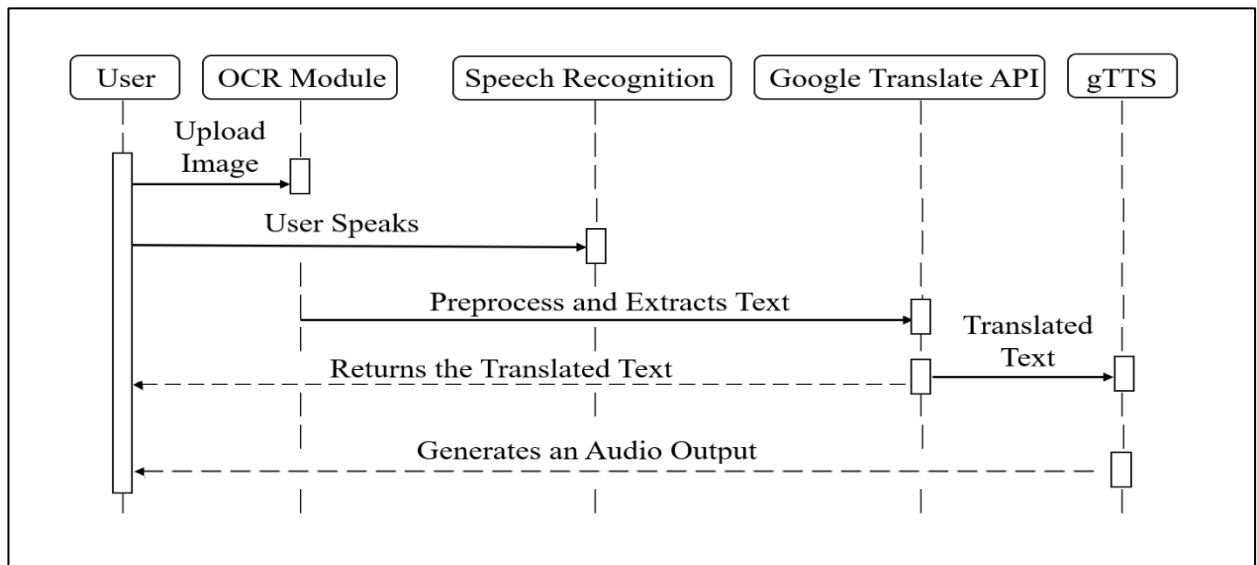


Fig 3.1.5: Sequence Diagram

3.1.6. Activity Diagram

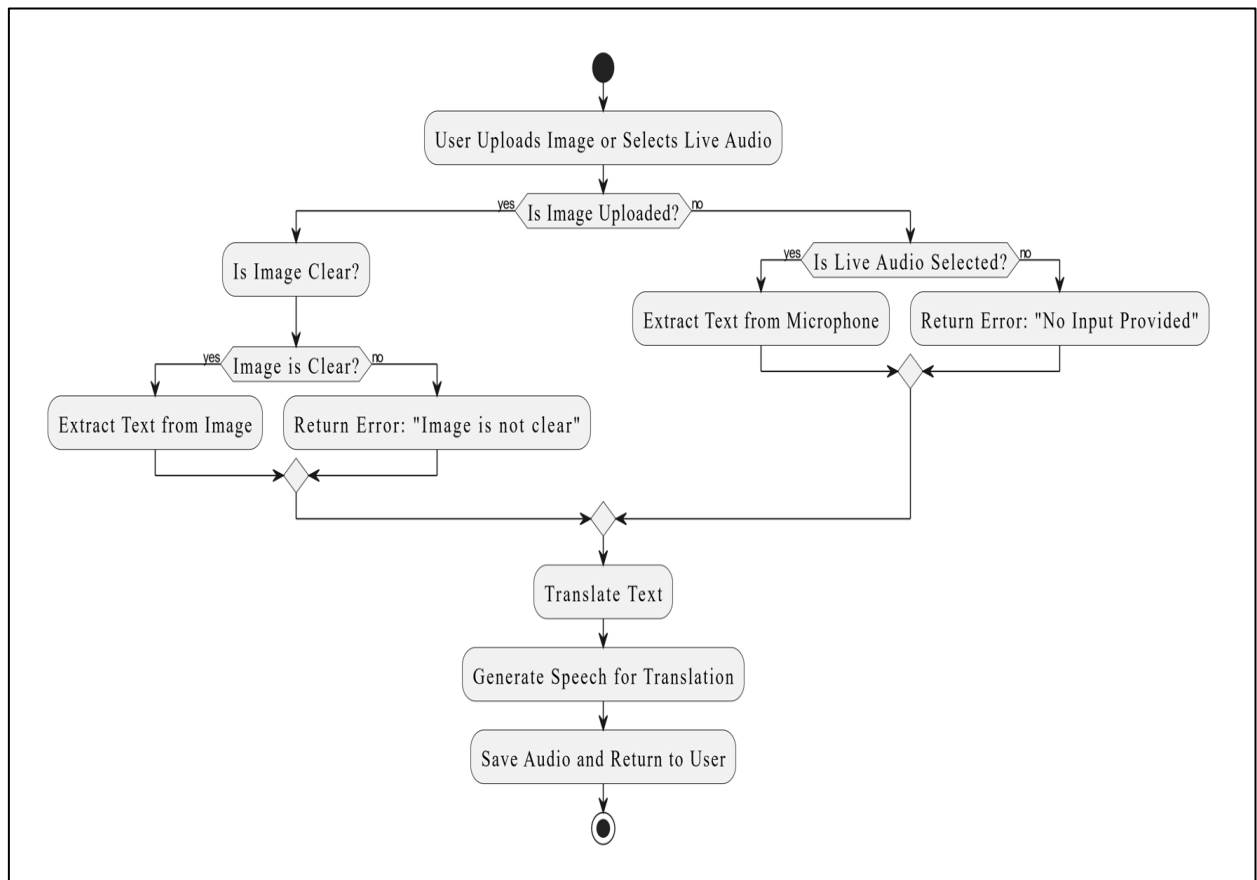


Fig 3.1.6: Activity Diagram

3.2. TECHNOLOGIES USED

3.2.1. Python

Python is a high-level, general-purpose programming language known for its simple syntax and readability, which makes it a popular choice among developers and researchers alike. Created by Guido van Rossum in the early 1990s, Python emphasizes code readability and supports object-oriented, procedural, and functional programming paradigms. Its versatility allows it to be used across various fields such as web development, data science, automation, artificial intelligence, and machine learning.

One of Python's greatest strengths is its comprehensive standard library and its vast ecosystem of third-party packages available via the Python Package Index (PyPI). These libraries provide ready-to-use solutions for tasks like file handling, networking, data visualization, machine learning, natural language processing, and more. Python is also cross-platform, meaning code written on one operating system can often run on others without modification.

Python integrates seamlessly with other technologies, making it ideal for building scalable web applications, scripting tasks, and developing backend services. It's the foundation for many powerful frameworks and tools such as Flask for web development, TensorFlow for machine learning, and OpenCV for computer vision.

In educational and research settings, Python is especially favored due to its simplicity, which allows students to focus on programming logic without getting bogged down by complex syntax. This makes it an ideal choice for prototyping and academic projects.

Overall, Python serves as the core of many modern applications and is an essential tool for developers working in today's technology-driven world. Its ease of use and strong community support continue to make it one of the most popular programming languages globally.

3.2.2. Flask

Flask is a lightweight web framework written in Python and is often referred to as a "micro-framework" because it does not require particular tools or libraries. It was created by Armin Ronacher and is built on top of the Werkzeug WSGI toolkit and the Jinja2 templating engine. Flask is designed to make getting started quick and easy, while still offering the flexibility to scale to complex applications.

One of Flask's key strengths is its simplicity and minimalism. Unlike full-stack frameworks like Django, Flask gives developers full control over the architecture of their web applications. It follows a modular design, allowing components such as databases, authentication, and form handling to be added as needed. This makes Flask especially suitable for small- to medium-sized projects or for prototyping.

Flask supports URL routing, request and response handling, template rendering, and session management. It allows the developer to define different routes (URLs) and link them to specific Python functions, creating a responsive and dynamic user experience. The use of Jinja2 templates allows dynamic data to be inserted into HTML, making the frontend responsive to backend data.

Flask also provides an easy way to serve static files such as CSS, JavaScript, and images, making it suitable for web apps with interactive frontends. It can also integrate easily with frontend technologies like AJAX, allowing asynchronous operations such as file uploads or data updates.

In the context of this project, Flask serves as the backbone of the application, managing user requests like image uploads or voice inputs, processing them on the backend, and returning results in the form of translated text or speech audio—all through simple and efficient routing.

3.2.3. OCR & Tesseract OCR

OCR, or Optical Character Recognition, is the technology used to identify and extract text from images, scanned documents, or any non-editable sources. It plays a crucial role in converting physical or visual data into machine-readable text. OCR technology is widely used in applications like digitizing printed documents, reading signboards, license plates, and assisting visually impaired users.

Tesseract OCR is an open-source OCR engine initially developed by Hewlett-Packard and currently maintained by Google. It is considered one of the most accurate free OCR engines available. Tesseract supports multiple languages, and additional training can be done to recognize custom fonts or handwriting. It can output text in various formats, including plain text and hOCR (HTML-based format with positional data).

To extract text, Tesseract processes an image through several stages: image preprocessing (binarization, noise removal), layout analysis (detecting lines and words), character recognition, and post-processing (spell correction, formatting). In Python, Tesseract can be easily used with the pytesseract wrapper, which allows integration into applications for real-time OCR functionality.

For optimal results, the quality of the input image is crucial. Preprocessing techniques like resizing, converting to grayscale, thresholding, and denoising significantly enhance the accuracy of text recognition. These steps can be implemented using libraries like OpenCV before passing the image to Tesseract.

In this project, Tesseract is the tool responsible for converting the text on uploaded signboard images into readable text. This output is then passed on to the translation and text-to-speech components, creating a seamless flow from visual input to multilingual audio output.

3.2.4. OpenCV

OpenCV (Open-Source Computer Vision Library) is a powerful and widely used open-source computer vision and machine learning software library. Originally developed by Intel, it is now supported by a large community and open-source contributors. OpenCV provides a vast collection of functions for real-time image processing, computer vision, and machine learning tasks. It supports multiple programming languages, including Python, C++, and Java, with Python (cv2) being one of the most commonly used interfaces due to its simplicity and ease of integration with other tools and libraries.

The core strength of OpenCV lies in its ability to handle image and video analysis effectively. It can read, write, and manipulate images and video streams, detect shapes and edges, apply filters, and perform object detection and recognition. For applications like OCR (Optical Character Recognition), OpenCV can be used to pre-process images—resizing, grayscaling, thresholding, noise removal, and contour detection—to enhance the accuracy of text extraction.

In Python, OpenCV is accessed using the cv2 module, which allows developers to write concise and readable code for a wide range of tasks. Functions like cv2.imread(), cv2.cvtColor(), cv2.threshold(), and cv2.findContours() make it easy to process and manipulate images efficiently. OpenCV also supports integration with deep learning models, allowing developers to use pre-trained neural networks for facial recognition, object classification, and real-time detection.

OpenCV is especially useful in real-time applications where fast and efficient image processing is needed. It uses optimized algorithms to reduce processing time, which is vital in use cases like surveillance, robotics, autonomous vehicles, and interactive user interfaces. In projects involving OCR, like the multilingual signboard translator, OpenCV helps improve text extraction by enhancing image quality before it's passed to the OCR engine.

Another key benefit of OpenCV is its cross-platform support and compatibility with other popular libraries like NumPy, Tesseract, and TensorFlow. It can be used both in desktop and embedded environments (like Raspberry Pi), making it suitable for a wide range of computer vision projects. Overall, OpenCV is a robust and flexible library that provides all the essential tools needed to build powerful image-processing and vision-based applications.

3.2.5. Speech Recognition

Speech recognition technology enables machines to understand and transcribe spoken language into text. This natural interface allows users to interact with systems through voice commands, improving accessibility and user-friendliness. Speech recognition is commonly used in virtual assistants, dictation software, and smart home devices.

In Python, the most commonly used library for speech recognition is the speech_recognition module. It supports multiple speech recognition engines, including Google Web Speech API, IBM Speech to Text, and CMU Sphinx. The library allows developers to process audio from files or directly from the microphone in real time, converting it into plain text with high accuracy.

Using speech recognition involves capturing audio input, typically via a microphone, converting it to WAV format if necessary, and then sending it to a speech recognition engine for transcription. The result is returned as a string, which can be used for further processing, such as translation or command execution.

One of the key challenges in speech recognition is handling noisy backgrounds or accents, which can lead to inaccuracies. Enhancing microphone sensitivity and using speech enhancement algorithms can help improve results. The Google Speech API, in particular, offers robust results and supports multiple languages and dialects.

In this project, this technology is used to capture and convert live speech input from the user. The recognized text is then sent to the Google Translate API, and the resulting translation is turned into audio using gTTS, enabling a complete voice-based translation experience.

3.2.6. Google Translate API

Google Translate API is a cloud-based service that enables dynamic translation of text between over 100 supported languages. It is part of Google Cloud's suite of machine learning services and is used globally in applications ranging from chat apps to multilingual content platforms and accessibility tools.

The API provides powerful features, including automatic language detection, batch translation, and support for HTML content. Developers can access the API through RESTful HTTP requests or using client libraries in languages like Python. The response is usually returned in JSON format, containing translated text and metadata.

In Python, developers often use the googletrans library (a free and unofficial client) or make direct HTTP requests with appropriate credentials if using the official paid API. The API is capable of translating entire documents, web pages, or single strings instantly and with high accuracy, thanks to Google's neural machine translation model.

A major advantage of using Google Translate is its continuous learning. The system improves over time by learning from user interactions and community contributions, making it increasingly reliable for both casual and formal translations.

In this project, the Google Translate API enables the conversion of OCR or voice-extracted text from one language to another. This makes it possible for users to understand foreign signboards or spoken phrases in their preferred language, facilitating cross-language communication in real time.

3.2.7. Google Text-to-Speech API

gTTS (Google Text-to-Speech) is a Python library and command-line utility that interfaces with Google's Text-to-Speech API to convert text into spoken audio. It's designed to provide a

simple and easy-to-use solution for adding speech synthesis to Python applications.

The library supports multiple languages and regional dialects, allowing developers to choose from a variety of voices. The text input is processed by Google's speech synthesis engine, and the output is returned as an .mp3 audio file, which can be played back or saved for future use.

Using gTTS involves passing a string of text along with the target language code (e.g., 'en' for English, 'hi' for Hindi), and invoking the save() function to generate the audio file. This output can be embedded into web applications using HTML audio tags.

gTTS is particularly useful in accessibility applications where users benefit from audio output, such as visually impaired individuals or language learners. It also adds an interactive layer to web and mobile apps, enabling a more engaging user experience.

In the multilingual translator project, gTTS is used to convert the translated text into an audio message in the selected language. This enables users not only to read the translation but also to hear it, making the tool especially useful for travelers, tourists, and multilingual communication scenarios.

3.2.8. HTML/CSS/JavaScript

HTML (HyperText Markup Language) is the foundation of all web pages. It provides the structure and layout of a website using elements such as headings, paragraphs, buttons, forms, and images. Each HTML element acts as a building block, and together they define the content and structure of a web page.

CSS (Cascading Style Sheets) is used to control the appearance and layout of HTML elements. With CSS, developers can define colours, fonts, margins, padding, grid layouts, animations, and responsiveness. It enables consistent design across multiple pages and enhances the visual appeal of web applications.

JavaScript is a scripting language used to add interactivity to web pages. It allows real-time actions like validating user inputs, updating content without reloading the page (AJAX), handling button clicks, form submissions, and even manipulating media elements. JavaScript is essential for creating dynamic, user-responsive applications.

Together, HTML, CSS, and JavaScript form the core trio of web development. HTML defines the content, CSS styles it, and JavaScript makes it interactive. These technologies are also supported by powerful frameworks and libraries like Bootstrap, React, Vue, and Tailwind CSS.

In the Flask-based application, HTML is used for creating the user interface, CSS for styling the layout (such as buttons, language dropdowns, and text boxes), and JavaScript for controlling behaviours like image preview, audio playback, and possibly live voice recording. This combination ensures a responsive and user-friendly frontend for the backend logic.

3.3. SYSTEM CONSTRAINTS

1. **Performance:** Real-time performance is a critical requirement. OCR processing can be slow depending on the complexity of the image, which may impact the user experience if not optimized.
2. **Accuracy of OCR:** OCR accuracy depends on the quality of the image. The system may struggle with noisy or unclear images.
3. **Limited Language Support in TTS:** The gTTS API supports a wide range of languages but does not support all dialects or regional variations, which might impact the accuracy of speech synthesis.
4. **Image Size:** Large image files might slow down processing or require resizing before OCR can be applied effectively.

3.4. HARDWARE AND SOFTWARE REQUIREMENTS

The hardware and software requirements for the multilingual signboard translator system are designed to ensure smooth functioning, ease of development, and scalability. The system involves processing images, running OCR, performing translations, and generating speech, all of which have specific hardware and software dependencies.

3.4.1. Hardware Requirements

- **Processor:** Minimum 2 GHz quad-core CPU; faster processors improve OCR and translation performance.
- **RAM:** At least 8 GB of RAM is recommended to handle OCR, image processing, and concurrent user actions smoothly.
- **Storage:** Minimum 200 MB free space to store uploaded images and generated audio files.
- **Internet:** Stable connection (10 Mbps or higher) needed for real-time API calls to Google Translate and gTTS.

3.4.2. Software Requirements

- **OS:** Compatible with Windows, Linux, and macOS; 64-bit architecture recommended.
- **Python:** Version 3.7 or higher; required to run Flask and integrate OCR, translation, and speech modules.
- **Flask:** Used for building the web application's backend and managing routes, requests, and rendering.
- **Browser:** Works on modern browsers like Chrome, Firefox, Edge, and Safari with a responsive design.
- **Tesseract OCR:** Must be installed along with language training files for text recognition.

3.4.3. Google APIs

- **Translate API:** For multilingual text translation (requires API key).
- **gTTS API:** For converting translated text to speech (also requires API access)

4. MODULES AND IMPLEMENTATION

The multilingual signboard translator system is comprised of several modular components, each responsible for a specific function within the system. These components are designed to work together in a seamless workflow, starting from user input to generating the final translated and audio output.

4.1. IMAGE INPUT MODULE

The image input module is a crucial component as it allows the user to upload images containing signboard text. This module is designed to provide a smooth user experience by integrating an easy-to-use interface for uploading images. Using HTML and JavaScript, the frontend provides a form that allows users to select an image file from their local device. The image file is then sent to the Flask server using HTTP POST requests.

Once the image is uploaded, the backend performs validation to ensure that the file is of an acceptable format, typically JPG, PNG, or TIFF, as these are commonly used for signboard images. The server checks the file type and the size to prevent unnecessary load on the system. If the image fails the validation, the user receives an appropriate error message, prompting them to upload a valid file. To improve processing time and handle larger files, the image is often resized or compressed before passing it to the next stage.

After the image is successfully uploaded, it is stored in the server's static/uploads/ folder. The Flask route that handles this upload ensures that the file is saved securely with a unique name to avoid conflicts. The file is now ready for processing by the next module—OCR text extraction.

The module also provides users with feedback during the image upload process, such as progress indicators or status messages. This is important to keep users informed in case the image file is large or requires additional time to process. The system's image input module ensures accessibility by offering clear instructions and validation prompts to guide users through the uploading process.

In the case of errors, such as a corrupted or unreadable image, the module provides helpful feedback. It instructs the user to upload a clearer image or suggests ways to improve image quality for better OCR performance. This is crucial because poor image quality can lead to inaccurate text extraction, which impacts the overall translation accuracy.

4.2. AUDIO INPUT MODULE

The audio input module introduces a key feature of the multilingual signboard translator, enabling users to provide live voice input for translation. This module is particularly valuable for users who may find it more convenient to speak rather than upload an image. Using HTML5 and JavaScript, the system captures the user's voice via the device's microphone. The browser's Web API is used to interact with the microphone and record audio, providing a seamless and real-time interaction.

Once the user speaks into the microphone, the audio is processed on the server side using Python's SpeechRecognition library. This library supports several speech-to-text engines, including Google Web Speech API, and is used to transcribe the speech into text. The transcribed text is then passed to the translation module, where it is translated into the user's selected language.

The audio input module also includes error handling, which is particularly important for speech recognition. It can handle cases where the speech is too soft, unclear, or interrupted by noise. In these cases, the system asks the user to speak more clearly or repeat their input. This feedback mechanism ensures that the system remains user-friendly and accommodating even in less-than-ideal conditions.

Additionally, the system allows users to select between "Upload Image" and "Speak Now" input options through a simple toggle on the frontend interface. This flexibility ensures that the user experience is versatile and can accommodate different preferences or situations. For example, users in noisy environments might prefer using images instead of speaking, while others might find speaking more efficient.

Finally, once the speech is successfully converted to text, it is passed through the translation pipeline, where it undergoes the same processing steps as the image-extracted text. This ensures a uniform experience, whether the user uploads an image or speaks directly to the system.

4.3. OCR TEXT EXTRACTION

The OCR text extraction module is at the heart of the system's image processing capability. Using Tesseract OCR, an open-source OCR engine, the module analyzes the image to extract the text from the signboard. Tesseract works by analyzing the pixels of the image, detecting shapes and patterns that resemble characters, and comparing these shapes to a trained model of known characters.

Before sending the image to Tesseract, it undergoes preprocessing to improve the OCR accuracy. This preprocessing might include converting the image to grayscale, adjusting contrast, or thresholding to distinguish text from the background. By using Python's OpenCV library, the image is enhanced to remove noise, straighten text, and highlight the regions where the text is likely located.

After the preprocessing, the image is passed to Tesseract, which begins its text extraction process. Tesseract identifies letters and words, turning them into machine-readable text. It also tries to recognize fonts and styles used in the signboard, though this can sometimes lead to inaccuracies, especially with unconventional fonts or poor image quality.

Once the OCR process is complete, the text is returned as a string, which is then sent to the next module for translation. The system's accuracy in text extraction depends heavily on the quality of the image. For this reason, the user is advised to upload clear and well-lit images for optimal OCR results.

If the system cannot extract any text, it notifies the user that the image quality is insufficient for OCR processing and prompts them to upload a clearer image. This ensures that the system maintains high standards of accuracy and usability, and it prevents users from receiving incomplete or incorrect translations.

4.4. LANGUAGE DETECTION AND TRANSLATION

Once the text is extracted from the image or audio input, the next step is translation. Google Translate API is used for this task, which supports translation across more than 100 languages. The system first detects the source language of the extracted text, and this is done automatically by Google Translate. The user can also manually select the target language from a dropdown menu provided in the frontend.

The system sends the extracted text to the Google Translate API, which processes the text and returns the translated version. The translation is fast, usually taking only a few seconds, but the speed can vary depending on the complexity of the text and the languages involved. Once the translation is received, it is displayed to the user on the web interface.

Language detection is an essential feature for systems that handle multilingual content. By automatically identifying the source language, the system simplifies the user's interaction. This means users don't need to manually select the original language, streamlining the translation process. Additionally, Google Translate's broad language support ensures that the system can cater to a wide range of users from different linguistic backgrounds.

The translation module also includes error handling, such as managing situations where translation fails or when the source text is not recognized. If the translation process encounters issues, the system provides an error message and asks the user to try again with different input.

Once the text has been translated, it is passed to the next module, where it will be converted into speech. The translation module also includes an option for users to view the translated text, which is particularly helpful in cases where the user needs to verify the translation before listening to the audio output.

4.5. TEXT-TO-SPEECH CONVERSION

The text-to-speech (TTS) conversion module plays a key role in making the system accessible to users who might prefer to listen to the translated text. This module uses the Google Text-to-Speech (gTTS) API to convert the translated text into audio. gTTS supports a wide variety of languages and can synthesize speech in different accents and tones.

Once the translated text is received, it is passed to the gTTS API, which generates an MP3 audio file. This audio file is saved in the server's static/audio/ directory, and the user can listen to it directly through the web interface. The generated audio is typically clear and natural-sounding, making it a useful tool for individuals who need spoken translations.

The system ensures that the audio is generated in the language selected by the user, providing an additional layer of customization. Users can also adjust the speech speed or pitch if desired, further enhancing the accessibility and usability of the application.

This module helps users who may be visually impaired or those who find it easier to listen to translations rather than read them. The combination of OCR, translation, and TTS features in this module provides a comprehensive solution for signboard translation.

4.6. FRONTEND DESIGN AND USER INTERACTION

The frontend of the multilingual signboard translator system is designed to be simple, intuitive, and responsive. It is built using HTML, CSS, and JavaScript, with Bootstrap ensuring that the interface is responsive and adapts to various screen sizes. The frontend is responsible for handling user interactions, guiding them through the process of input, and displaying the results.

The user is greeted with two primary options: "Upload Image" and "Speak Now". These options are presented as buttons on the interface, and the user can choose either to upload an image of a signboard or speak directly into the microphone. The frontend includes feedback mechanisms to show the user the status of their input, such as displaying loading indicators when the image is being processed or when the translation is being generated.

Once the OCR and translation processes are complete, the results are displayed in a readable format. The translated text is shown in a box on the page, and the user is given an option to listen to the audio translation. The interface is designed to be straightforward, with minimal distractions, ensuring that the focus remains on the core functionality of the system.

Error handling is also incorporated into the frontend design, ensuring that users are alerted if there is an issue with the input (e.g., no text detected in the image or unclear speech). This prevents users from becoming frustrated and helps them resolve issues quickly.

The frontend also includes options to customize the user experience, such as adjusting the volume or language preferences. This makes the system more flexible and adaptable to different user needs.

4.7. FLASK BACKEND INTEGRATION

The backend of the system is built using the Flask web framework. Flask handles all the server-side functionality, including receiving user input, processing the images and audio, managing API requests (to Tesseract OCR, Google Translate, and gTTS), and sending the appropriate responses to the frontend.

Flask routes are designed to handle specific user requests, such as receiving an image upload or processing audio input. The backend also ensures that data is passed between the different modules efficiently, ensuring that the OCR output is translated and then converted into speech.

Flask's routing system is used to direct incoming requests to the appropriate modules. It manages the communication between the frontend and backend, handling file uploads, serving static files (such as the generated audio), and ensuring smooth interaction between the user and the system. The backend also includes error handling to provide feedback to users if any issues arise during the processing of their input.

5. TESTING AND EVALUATION

Testing and evaluation are critical steps in ensuring the system’s reliability, functionality, and user satisfaction. The testing approach for the multilingual signboard translator includes several stages, including unit testing, integration testing, and performance evaluation.

5.1. TEST STRATEGY

The test strategy of this project focuses on verifying that each module of the system performs as expected and integrates seamlessly with other components. The approach includes:

- Unit testing of individual modules such as the image input, OCR, translation, and text-to-speech components.
- Integration testing to ensure that the modules work together without errors, such as testing the end-to-end flow from image upload to audio output.
- Performance testing to evaluate the speed and responsiveness of the system, especially during image processing and translation.
- User acceptance testing to ensure that the system meets user expectations in terms of usability, functionality, and overall experience.

5.2. SAMPLE TEST CASES

Test ID	Test Case ID	Module	Input Provided	Expected Output	Actual Output	Status
1	TC001	OCR	English Signboard Image	Extracted text: “WELCOME”	Extracted text: “WELCOME”	Pass
	TC002	Translation	Extracted text to Hindi	Translated text: “स्वागत”	Translated text: “स्वागत”	Pass
	TC003	TTS	Translated Text	Audio file generated	The audio file plays	Pass
2	TC004	OCR	Hindi Signboard Image	Extracted text: “मुंबई”	Extracted text: “मुंबई”	Pass
	TC005	Translation	Extracted text to Tamil	Translated text: “மும்பை”	Translated text: “மும்பை”	Pass
	TC006	TTS	Translated Text	Audio file generated	The audio file plays	Pass

Test ID	Test Case ID	Module	Input Provided	Expected Output	Actual Output	Status
3	TC007	Live Audio	The user speaks	Extracted text: “Welcome to multilingual signboard translation”	Extracted text: “welcome to multilingual signboard translation”	Pass
	TC008	Translation	Extracted text to Hindi	Translated text: “బహుభాషా సైన్ బోర్డ్ అనువాదానికి స్వాగతం”	Translated text: “బహుభాషా సైన్ బోర్డ్ అనువాదానికి స్వాగతం”	Pass
	TC009	TTS	Translated Text	Audio file generated	The audio file plays	Pass
4	TC010	OCR	Telugu Signboard	Extracted text: “హైదరాబాద్”	No text is detected	Fail

Table 5.2: Test Cases

5.3. LOG OUTPUTS

Test Case 1:

Image uploaded successfully.

Extracted Text: “Welcome”

Selected Language: Hindi

Translated Text: “स्वागत”

Audio file saved at: static/audio/output.mp3

Test Case 2:

Image uploaded successfully.

Extracted Text: “முன்”

Selected Language: Tamil

Translated Text: “முன்பை”

Audio file saved at: static/audio/output.mp3

Test Case 3:

Listen to the audio successfully.

Extracted Text: “Welcome to multilingual signboard translation”

Selected Language: Telugu

Translated Text: “బహుభాషా సైన్ బోర్డ్ అనువాదానికి స్వాగతం”

Audio file saved at: static/audio/output.mp3

6. SCREENSHOTS

The user interface (UI) of the multilingual signboard translator has been designed to be intuitive and user-friendly, ensuring a smooth interaction for users of all technical backgrounds. Below, we describe the key components of the interface, including the upload and speak interface, OCR output and translation display, audio output functionality, and error handling screens.

6.1. WEB INTERFACE

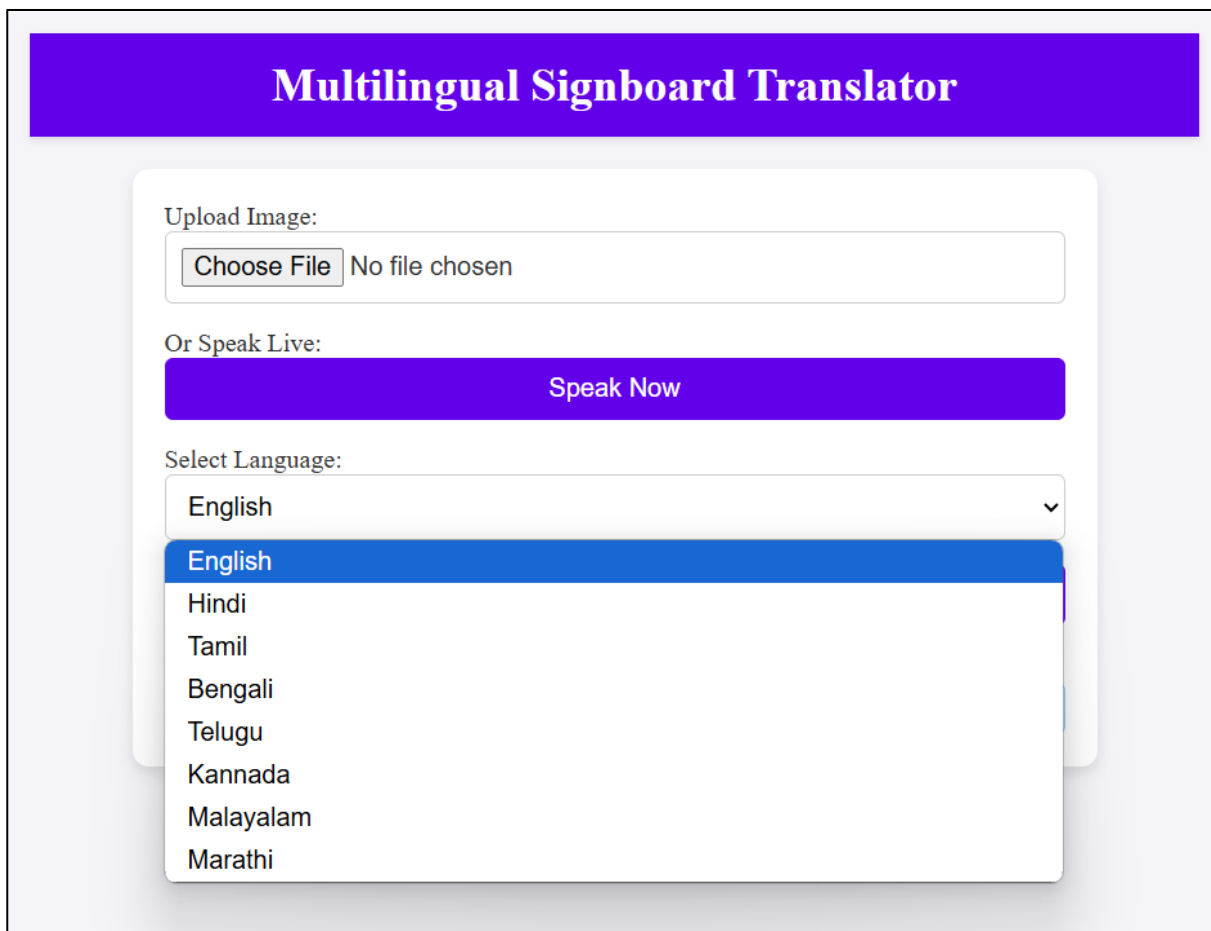


Fig 6.1: Web Interface

The first screen users encounter offers two input options: "Upload Image" and "Speak Now". These two options provide flexibility depending on the user's preferences and the situation. The "Upload Image" button allows users to upload an image file directly from their device, while the "Speak Now" button enables users to speak into their microphone for immediate voice input.

Prominently displayed on the bottom section is a dropdown menu listing eight supported languages like English, Hindi, Tamil, Bengali, Telugu, Kannada, Malayalam, and Marathi. This menu allows users to select their desired target translation language before submitting any input. The system currently supports major languages to facilitate broader usability across multilingual regions, especially in countries like India, where regional languages vary significantly. The

positioning of this element ensures it is visible, encouraging users to make a selection before proceeding.

The design is minimalist, with clear, bold buttons to help users easily navigate the application. Once a button is clicked, an appropriate action is initiated—either a file upload dialogue box appears for image uploads or the browser requests microphone access for speech input. Additionally, a progress bar or loading indicator is displayed to inform users about the current operation status, whether it’s image uploading, speech processing, or translation.

To improve user experience on mobile devices, the interface is fully responsive, adapting to smaller screens and making sure all options remain easily accessible. The design is compatible with various devices like tablets and smartphones. This responsiveness is key to making the tool versatile and ensuring that it functions well on both desktop and mobile platforms.

6.2. IMAGE UPLOADS

6.2.1. English Signboard Upload



Fig 6.2.1: Signboard 1

This signboard features the word “Welcome” prominently displayed in English, using a clean and bold typeface. Such signs are commonly used at entrances of commercial establishments, educational institutions, tourist centers, and airports to greet visitors. The image chosen for this use case has high contrast and good lighting, ensuring that the OCR module can accurately extract the text. It serves as a straightforward example of English-language recognition and is ideal for illustrating the core OCR process.

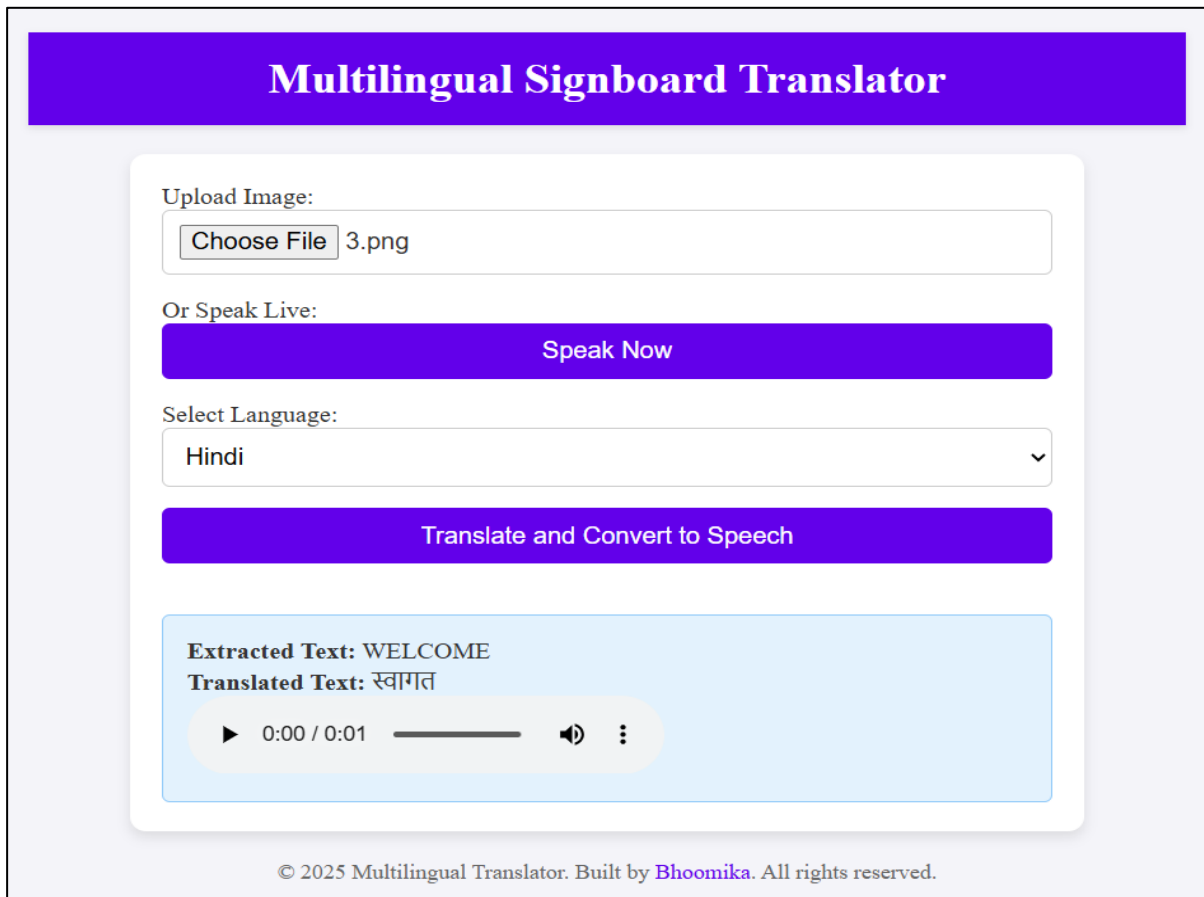


Fig 6.2.1.1: Translation of English Signboard ("Welcome") to Hindi

In this screen, the user has selected the “Upload Image” option and submitted an image of a signboard that contains the word “Welcome” written in English. Upon uploading, the image is processed by the backend using the Tesseract OCR engine, which extracts the visible text from the image. This extracted text is then displayed on the interface in a dedicated section under “Extracted Text.”

After the OCR stage, the system automatically translates the extracted text into Hindi, as selected by the user in the language dropdown. The translated result “स्वागत” is shown under the label “Translated Text”, offering a clear distinction between the original and the converted content. The interface also includes a “Listen to Translation” button, which uses Google Text-to-Speech (gTTS) to generate a spoken version of the translated Hindi text, enhancing accessibility for visually impaired users or those who prefer audio.

The layout for this result screen is visually appealing and informative. It provides feedback in real time, indicating when text is being processed or translated. If the image is blurry or of poor quality, the system also has mechanisms to notify the user with an appropriate error message or suggestion, such as uploading a clearer image for better OCR results. This helps users improve their input for accurate translation.

This screen highlights the core functionality of the application—processing a static image and converting its text to another language seamlessly. The clear display of each stage’s result (OCR and translation) and the ability to hear the translation make this a powerful and user-friendly feature for navigating multilingual environments.

6.2.2. Hindi Signboard Upload



Fig 6.2.2: Signboard 2

This image shows a real-world signboard with the word “मुंबई” (Mumbai) written in the Devanagari script. It reflects the kind of signage commonly seen at railway stations, bus stops, or highway indicators in India. The image is a strong representation of the system's ability to process native scripts, which are often more challenging than Latin-based ones due to their complex characters and ligatures. The clarity and font size in the image were crucial for successful OCR extraction using Tesseract.

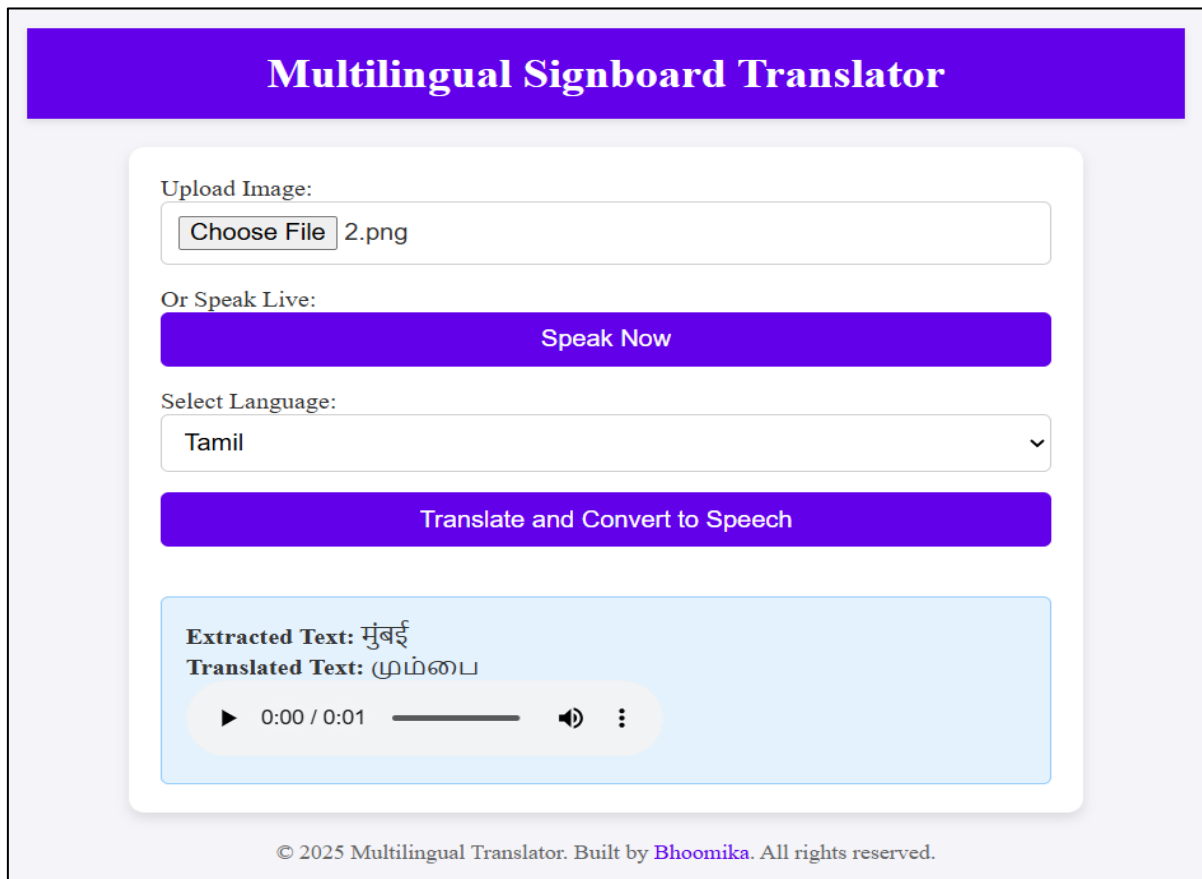


Fig 6.2.2.1: Translation of Hindi Signboard to Tamil

The screen captures a scenario where a Hindi-language signboard is uploaded for translation. The uploaded image contains the word “मुंबई” (Mumbai). This use case demonstrates the system’s ability to handle non-English inputs, showcasing the OCR engine’s flexibility in recognizing and processing multiple regional scripts, including Devanagari.

After extracting the Hindi text, the translation module converts the content into Tamil, based on the user’s selection. The translated Tamil text “மும்பை” appears next to the original Hindi, allowing users to compare both versions easily. This feature is especially helpful for individuals traveling across different Indian states where signboards may be written in unfamiliar regional languages. The user interface ensures both scripts are displayed legibly and clearly labeled for easy identification.

The system also provides the translated result in audio form, again using the gTTS API to convert the Tamil text into a clear audio clip. The inclusion of audio output significantly enhances usability, especially for users who may not be fluent in reading regional scripts. The playback controls are simple, including play and pause options for convenience.

This screen emphasizes the multilingual and inclusive nature of the application. It proves that the system is not limited to just English inputs but is capable of bridging communication gaps between regional languages, making it a versatile tool for both locals and travelers navigating different linguistic regions.

6.3. SPEAK LIVE

The screenshot displays the 'Multilingual Signboard Translator' interface. At the top, a purple header contains the title. Below it, the 'Upload Image' section has a 'Choose File' button and 'No file chosen' text. The 'Or Speak Live' section features a prominent purple 'Speak Now' button. Underneath, the 'Select Language' dropdown is set to 'Telugu'. A large purple button labeled 'Translate and Convert to Speech' is positioned below the language selection. The bottom section, highlighted in light blue, shows the 'Extracted Text' as 'welcome to multilingual sign board translation' and the 'Translated Text' in Telugu: 'బహుభాషా సైన్ బోర్డు అనువాదానికి స్వాగతం'. Below the text is an audio player with a play button, a progress bar at 0:00 / 0:03, a speaker icon, and a menu icon. At the very bottom, a copyright notice reads: '© 2025 Multilingual Translator. Built by Bhoomika. All rights reserved.'

Fig 6.3: Translation of the Live Audio Input

In the above screen, the user opts to use the speech input feature by clicking the “Speak Now” button. The browser prompts the user to allow microphone access, after which the user speaks the sentence: “Welcome to multilingual signboard translation”. The system utilizes a speech recognition engine to convert the spoken input into text.

Once the text is transcribed, the translation module immediately processes it, translating it into Telugu, as chosen in the dropdown. The translated Telugu text is then displayed in the interface as “బహుభాషా సైన్ బోర్డు అనువాదానికి స్వాగతం”. The integration between voice recognition, translation, and display is seamless, creating an efficient experience for users who may prefer voice interaction or are unable to type or upload images.

The translated text is also fed into the gTTS module, which generates an audio file in Telugu. Users can listen to the translation directly via the playback controls embedded in the interface. The audio is clear and natural, enhancing understanding and accessibility for users who prefer spoken output or are learning the target language.

This screen effectively demonstrates the system’s speech-to-speech translation, highlighting its ability to accept spoken language, translate it, and return the output both in text and audio. It opens up new possibilities for hands-free communication and is especially useful in real-time translation scenarios, such as guiding travelers or translating announcements.

7. RESULTS AND ANALYSIS

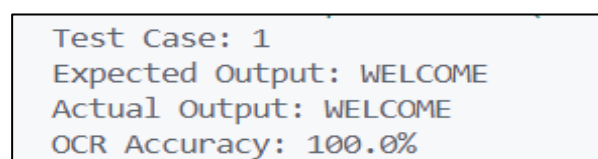
The evaluation of the multilingual signboard translator involves assessing several key aspects, including the accuracy of OCR detection, the quality of translations, the effectiveness of clarity verification, and user feedback. The results from these tests provide a comprehensive view of the system's performance.

7.1. ACCURACY OF OCR DETECTION

The accuracy of the OCR module is one of the most critical aspects of the system, as it directly influences the quality of the translated text. OCR accuracy depends heavily on the quality of the input image. In ideal conditions, where the image is clear and the text is legible, Tesseract OCR demonstrates high accuracy in text extraction, often achieving recognition rates above 90%, and in some cases, above 95%.

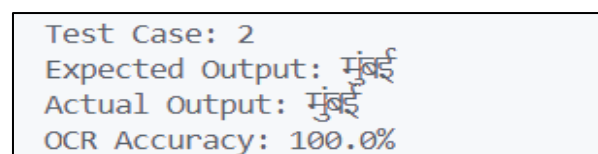
However, OCR accuracy can drop significantly in cases of low-quality images, such as those with poor lighting, noisy backgrounds, or distorted text. In such cases, the system provides feedback to the user, urging them to upload a clearer image for better results. Several preprocessing techniques, such as grayscale conversion and noise reduction, are applied to improve the OCR accuracy in suboptimal conditions.

During testing, the system was able to extract text accurately from a variety of signboards, including those with different fonts and languages. The results indicate that Tesseract OCR is quite effective for standard fonts, though it may struggle with highly stylized or handwritten text. OCR accuracy is crucial as any error in text extraction directly impacts the subsequent translation process.



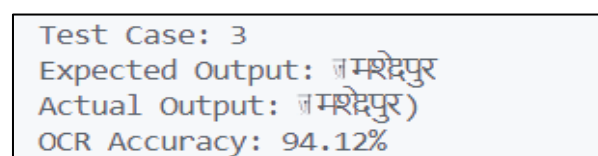
```
Test Case: 1
Expected Output: WELCOME
Actual Output: WELCOME
OCR Accuracy: 100.0%
```

Fig 7.1: Test Case-1



```
Test Case: 2
Expected Output: मुंबई
Actual Output: मुंबई
OCR Accuracy: 100.0%
```

Fig 7.1.1: Test Case-2



```
Test Case: 3
Expected Output: महेशपुर
Actual Output: महेशपुर)
OCR Accuracy: 94.12%
```

Fig 7.1.2: Test Case-3

7.2. TRANSLATION QUALITY

The translation quality is largely determined by the effectiveness of the Google Translate API. During tests, the translations were generally accurate, particularly for widely spoken languages. The quality of the translations depends on factors such as sentence structure, idiomatic expressions, and the complexity of the original text.

For simple phrases and common languages, the translations were nearly flawless, providing accurate and contextually appropriate translations. However, more complex sentences or less common languages occasionally resulted in slight inaccuracies or unnatural phrasing. The system allows users to review both the original and translated text, giving them the opportunity to verify the accuracy of the output.

In some cases, the translation module may face challenges with colloquial phrases or specialized terminology. This is an area where future enhancements, such as incorporating a custom translation model, could improve the system's performance, particularly in domain-specific contexts such as technical or legal translations.

7.3. CLARITY VERIFICATION

Clarity verification is a critical step in ensuring that the OCR module produces readable and usable text. The system includes features to assess image quality before performing OCR, helping to reduce errors caused by unclear or blurry images. If the image quality is insufficient, the user is informed and asked to upload a better image.

In tests, when the image quality met the required standards, the OCR module performed excellently. However, in cases of low resolution or blurriness, the system failed to extract meaningful text, resulting in a prompt to retry with a better image. While the clarity verification feature does help mitigate some OCR issues, it remains a challenging aspect due to the inherent limitations of OCR technology.

7.4. USABILITY AND USER FEEDBACK

User feedback was collected through a series of usability tests, where participants interacted with the system and provided their experiences. The results indicated that the system was easy to use and intuitive, with users quickly learning how to upload images and use the speech input functionality.

The system's design, which includes clear instructions and feedback at every step, was particularly praised for its simplicity and accessibility. However, some users suggested that adding more language options and improving error handling during speech input could enhance the overall experience. Despite these minor issues, the system was well-received, and most users felt that the translation and audio output features were useful and effective.

8. CONCLUSION

The multilingual signboard translator has proven to be a highly effective tool for translating text from images and voice inputs. By integrating Optical Character Recognition (OCR) with machine translation (using the Google Translate API) and Text-to-Speech (TTS) functionalities, the system provides a seamless experience for users seeking multilingual support in real-time. The system effectively bridges the gap for people in environments where they need immediate translation of signs, billboards, or other text-based content. In terms of functionality, the system was able to accurately process images, extract text, translate it into various languages, and provide clear audio output—all within a simple and user-friendly interface. The inclusion of two input methods—image upload and speech input—ensures that the tool can be used in diverse real-world situations, whether the user has a clear image of a signboard or simply wants to translate something verbally. The interface design is minimal yet effective, allowing users to interact with the system effortlessly. It is clear, responsive, and provides real-time feedback on the status of each operation, which is crucial for maintaining user engagement and trust. Moreover, the accuracy of OCR and translation is satisfactory for most common use cases, and users have generally found the system's ability to translate content quickly to be highly useful. The multilingual signboard translator demonstrates significant potential for enhancing cross-lingual communication in public spaces, especially in multicultural urban settings, tourist areas, and transportation hubs. Overall, the project has successfully fulfilled its goal of providing an easy-to-use, real-time multilingual translation tool.

9. FUTURE ENHANCEMENTS

1. **More Languages & Dialects:** Expand support for regional, indigenous, and complex scripts for better accessibility.
2. **Real-time Camera Input Integration:** Allow users to translate signboards directly using a live camera feed instead of uploading an image, enabling real-time detection and translation on mobile or web platforms.
3. **Offline Functionality:** Integrate offline OCR and translation libraries to allow users to use the application in areas with limited or no internet access, improving accessibility.
4. **Public Transport & Tourism Integration:** Assist travelers and commuters with real-time sign translations.
5. **Accessibility Features:** Include features like screen readers, voice command navigation, and support for visually impaired users to ensure inclusive access to the system.
6. **Gesture Interaction:** Hands-free usability with gesture-based controls.
7. **Customizable Experience:** Personalize voice styles and accents, and add language learning features.
8. **Mobile App Deployment:** Port the application into a native mobile app (Android/iOS) using platforms like Flutter or React Native for better user convenience and portability.
9. **Language Dialect and Contextual Understanding:** Improve translation accuracy by incorporating dialect-specific nuances and context-aware translation using AI-based language models like BERT or GPT-based APIs.
10. **User Feedback and Ratings System:** Add an interface for users to provide feedback on translation quality and audio clarity, helping improve future versions of the system based on user insights.

10. REFERENCES

- [1] Khalid, Sara, et al. "A Robust Intelligent System for Text-Based Traffic Signs Detection and Recognition in Challenging Weather Conditions." *IEEE Access* (2024). J. Clerk Maxwell, A Treatise on Electricity and Magnetism, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.
- [2] Do, Tien, et al. "SignboardText: Text Detection and Recognition in In-the-Wild Signboard Images." *IEEE Access* (2024).
- [3] Panhwar, Muhammad A., et al. "Signboard detection and text recognition using artificial neural networks." 2019 IEEE 9th international conference on electronics information and emergency communication (ICEIEC). IEEE, 2019.
- [4] Chavre, Pooja, and Archana Ghotkar. "Scene text extraction using stroke width transform for tourist translator on android platform." 2016 International Conference on Automatic Control and Dynamic Optimization Techniques (ICACDOT). IEEE, 2016.
- [5] Xavier, Seles, and R. Reshmi. "Automatic detection and recognition of text in traffic sign boards based on word recognizer." *IJIRST–International Journal for Innovative Research in Science & Technology* 3.04 (2016).
- [6] Amenador, Kate Benedicta, and Zhiwei Wang. "The image of China as a destination for tourist: translation strategies of culture-specific items in the Chinese-English food menus." *SAGE Open* 13.3 (2023): 21582440231196656.
- [7] Usha, S. M., D. Mahesh Kumar, and H. B. Mahesh. "Traffic Signboard Recognition and Text Translation System using Word Spotting and Machine Learning." *ITM Web of Conferences*. Vol. 50. EDP Sciences, 2022.
- [8] Vaidya, Shreyas, et al. "Show Me the World in My Language: Establishing the First Baseline for Scene-Text to Scene-Text Translation." *arXiv preprint arXiv:2308.03024* (2023).
- [9] Al-Hashemi, Rafeeq Abdul Rahman A., and Shoroq Almamon Alsharari. "Instant Arabic translation system for signboard images based on printed character recognition." *International Journal of Machine Learning and Computing* 3.4 (2013): 384-388.
- [10] Dhar, Rituparna, and Saswati Mukherjee. "Android-based Text Reader for Partial Vision Impairment." 2018 5th IEEE Uttar Pradesh Section International Conference on Electrical, Electronics and Computer Engineering (UPCON). IEEE, 2018.
- [11] Fujitake, Masato. "Diffusionstr: Diffusion model for scene text recognition." 2023 IEEE International Conference on Image Processing (ICIP). IEEE, 2023.
- [12] Ponnappalli, Srinivasu Badugu1, Swetha, Sharia Zainab, and Rida Fatima. "Signboard Text Translator: An Essential Guide for Tourists." *Delving* (2023).
- [13] Pandiyan, S. Sundara, and Charles Kelvin. "A Translator for Indian Sign Boards to English using Tesseract and SEQ2SEQ Model." 2021 International Conference on Simulation, Automation & Smart Manufacturing (SASM). IEEE, 2021.
- [14] Dona Lal, Jothika Mohan, Renjisha T S, Saviya Davis, and Divya Mohan. "Improved Real time Signboard Detection and Translation Using FRCNN." *International Journal of Computing, Communications and Networking* Volume 10, No.2 (2021): 2319-2720.

- [15] Ajmal, Muhammad, et al. "Image to Multilingual Text Conversion for Literacy Education." 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA). IEEE, 2018.
- [16] Kavati, Ilaiah, et al. "Signboard text translator: a guide to tourist." International Journal of Electrical and Computer Engineering (IJECE) 7.5 (2017): 2496-2501.
- [17] Ramiah, Sathiapriya, Tan Yu Liong, and Manoj Jayabalan. "Detecting text based image with optical character recognition for English translation and speech using Android." 2015 IEEE student conference on research and development (SCOReD). IEEE, 2015.
- [18] Chen, Xilin, et al. "Automatic detection and recognition of signs from natural scenes." IEEE Transactions on image processing 13.1 (2004): 87-99.

APPENDIX

A. SAMPLE CODE

The following sections contain key code snippets for developing the multilingual signboard translator application. These snippets highlight the essential functionality for image processing, text extraction, translation, and text-to-speech conversion.

1. BACKEND CODE (app.py)

(i) Imports & Initialization

```
from flask import Flask, request, jsonify, render_template, send_file
import cv2
import pytesseract
import speech_recognition as sr
from googletrans import Translator
from gtts import gTTS
import os
import time

app = Flask(__name__)
```

(ii) Folder Setup

```
UPLOAD_FOLDER = 'static/uploads'
AUDIO_FOLDER = 'static/audio'
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(AUDIO_FOLDER, exist_ok=True)
```

(iii) Language Mapping for OCR

```
OCR_LANGUAGES = {'en': 'eng', 'hi': 'hin', 'ta': 'tam', 'bn': 'ben',
                  'te': 'tel', 'kn': 'kan', 'ml': 'mal', 'mr': 'mar'}
```

(iv) Image Clarity Check

```
def is_image_clear(image_path):
    image = cv2.imread(image_path)
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    fm = cv2.Laplacian(gray, cv2.CV_64F).var()
    return fm >= 100
```

(v) Extract Text from Image using OCR

```
def extract_text_from_image(image_path):
    extracted_texts = {}
    for lang_code, tesseract_lang in OCR_LANGUAGES.items():
        text = pytesseract.image_to_string(cv2.imread(image_path),
                                           lang=tesseract_lang).strip()
        if text:
            extracted_texts[lang_code] = text
    return extracted_texts
```

(vi) Extract Text from Live Audio

```
def extract_text_from_live_audio():
    recognizer = sr.Recognizer()
    with sr.Microphone() as source:
        print("Listening...")
        recognizer.adjust_for_ambient_noise(source)
    try:
        audio_data = recognizer.listen(source, timeout=5)
        text = recognizer.recognize_google(audio_data)
        return text
    except sr.UnknownValueError:
        return "Could not understand audio."
    except sr.RequestError:
        return "Speech recognition service unavailable."
```

(vii) Translate Text

```
def translate_text(text, target_language):
    translator = Translator()
    translated = translator.translate(text, dest=target_language)
    return translated.text if translated and translated.text.strip() else "Translation not available."
```

(viii) Convert Text to Speech

```
def text_to_speech(text, lang):
    tts = gTTS(text=text, lang=lang)
    filename = f"speech_{int(time.time())}.mp3"
    audio_path = os.path.join(AUDIO_FOLDER, filename)
    tts.save(audio_path)
    return filename
```

(ix) Main Route – / (GET and POST)

```
@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        target_lang = request.form.get('language', 'en') # ensure language is always
        captured

        if 'image' in request.files and request.files['image'].filename:
            file = request.files['image']
            filepath = os.path.join(UPLOAD_FOLDER, file.filename)
            file.save(filepath)

        else:
            return jsonify({"message": "No input provided. Please upload an image or speak
live."})

        translated_text = translate_text(extracted_text, target_lang)
        audio_filename = text_to_speech(translated_text, target_lang)
        audio_url = f"/audio/{audio_filename}"

        return jsonify({
            "detected_language": detected_lang,
            "extracted_text": extracted_text,
            "translated_text": translated_text,
            "audio_url": audio_url
        })

    return render_template('index.html')
```

(x) Serve Audio Files

```
@app.route('/audio/<filename>')
def get_audio(filename):
    return send_file(os.path.join(AUDIO_FOLDER, filename), as_attachment=True)
```

(xi) Run app.py

```
if __name__ == '__main__':
    app.run(debug=True)
```

2. FRONTEND CODE (index.html)

(i) Header

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Multilingual Signboard Translator</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: 'Times New Roman', serif;
      background-color: #f4f4f9;
      color: #333;
      display: flex;
      flex-direction: column;
      align-items: center;
      padding: 20px;
    }
    main {
      width: 100%;
      max-width: 600px;
      background: #fff;
      padding: 20px;
      border-radius: 10px;
      box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
      margin-top: 20px;
    }
    input, select, button {
      width: 100%;
      padding: 10px;
      margin-bottom: 15px;
      border: 1px solid #ccc;
      border-radius: 5px;
      font-size: 1rem;
    }
    button {
      background-color: #6200ea;
      color: #fff;
      border: none;
      cursor: pointer;
      transition: background-color 0.3s ease;
```

```

    }
    #result {
        margin-top: 20px;
        padding: 15px;
        background-color: #e3f2fd;
        border: 1px solid #90caf9;
        border-radius: 5px;
    }
    footer {
        margin-top: 20px;
        text-align: center;
        font-size: 0.9rem;
        color: #666;
    }
</style>
</head>

```

(ii) Body

```

<body>
  <header>
    <h1>Multilingual Signboard Translator</h1>
  </header>
  <main>
    <form id="uploadForm">
      <label for="image">Upload Image:</label>
      <input type="file" id="image" name="image" accept="image/*">

      <label for="liveAudioBtn">Or Speak Live:</label>
      <button type="button" id="liveAudioBtn">Speak Now</button>

      <label for="language">Select Language:</label>
      <select id="language" name="language">
        <option value="en">English</option>
        <option value="hi">Hindi</option>
        <option value="ta">Tamil</option>
        <option value="bn">Bengali</option>
        <option value="te">Telugu</option>
        <option value="kn">Kannada</option>
        <option value="ml">Malayalam</option>
        <option value="mr">Marathi</option>
      </select>
      <button type="submit">Translate and Convert to Speech</button>
    </form>

```

```

        <div id="result"></div>
    </main>
    <footer>
        <p>&copy; 2025 Multilingual Translator. Built by <a href="#">Bhoomika</a>.
        All rights reserved.</p>
    </footer>
    <script>
        document.getElementById("uploadForm").onsubmit = async (e) => {
            e.preventDefault();

            const imageSelected = document.getElementById("image").files.length > 0;

            if (!imageSelected && !liveAudioSelected) {
                alert("Please upload an image or speak live.");
                return;
            }

            if (liveAudioSelected) {
                liveAudioSelected = false;
                return;
            }

            function displayResult(result) {
                const resultDiv = document.getElementById("result");
                resultDiv.innerHTML = `
                    <p><strong>Extracted Text:</strong> ${result.extracted_text}</p>
                    <p><strong>Translated Text:</strong> ${result.translated_text}</p>
                    <audio
                        controls><source
                            src="${result.audio_url}"
                            type="audio/mpeg"></audio>
                `;
            }
        }
    </script>
</body>
</html>

```

B. ERROR LOGS AND RESOLUTIONS

Error Message	Cause	Resolution / Fix
TesseractNotFoundError: tesseract is not installed or it's not in your PATH	The Tesseract-OCR engine is either not installed or not accessible via the system PATH.	Installed Tesseract via OS package manager and added the full path in pytesseract.pytesseract.tesseract_cmd in app.py
FileNotFoundError: static/uploads/input.jpg	The file path specified does not exist due to either a missing upload or a filename mismatch.	Added validation checks for file existence and used secure_filename() to sanitize file names
gtts.tts.gTTSError: Failed to connect. Probable cause: Internet connectivity issue.	gTTS requires an internet connection to convert text to speech	Wrapped gTTS code in try-except block, added fallback message or user prompt to check internet
werkzeug.exceptions.BadRequest: 'file'	The form was submitted without selecting or uploading a file	Added frontend form validation and backend checks with if 'file' not in request. Files to gracefully handle missing uploads
UnicodeEncodeError: 'ascii' codec can't encode character	Special characters or non-English text are not handled properly	Ensured text encoding with UTF-8 in all string operations and file handling
googletrans.Translator().translate fails silently	Google Translate's free API endpoint is rate-limited or temporarily inaccessible	Implemented error handling using try-except and displayed user-friendly error messages on failure
ValueError: Audio file not found	The audio file was deleted or never created due to a previous failure	Checked if audio file exists before attempting playback; used os.path.exists()
SpeechRecognition.RequestError during live audio input	Audio recognition API failed to respond, or the audio was unclear	Added error-catching for the speech-to-text component and included retry instructions for the user
PermissionError: [Errno 13] Permission denied: 'output.mp3'	The application lacked write permission for saving the audio file	Ensured correct file permissions and used safe directories (static/audio/) with write access
cv2.error: OpenCV(4.x.x) :-1: error: (-5:Bad argument) in function 'imread'	OpenCV could not read the uploaded image (corrupt or unsupported format)	Validated file extensions before passing to OpenCV, limited to .jpg, .png, .jpeg

Table B.1: Error Logs and Fixes

QUALITY ANALYSIS AND ALIGNMENT OF STUDENT PROJECTS WITH PROGRAM OUTCOMES (POs) AND SUSTAINABLE DEVELOPMENT GOALS (SDGs)			
Course Name:	PROJECT WORK – II	AY	2024-25
Course Code:	1PW868AD	Semester	VIII
Name of the Guide:	Mrs. Unnati Khanapurkar	Class	AI&DS
PROJECT TITLE MULTILINGUAL SIGNBOARD TRANSLATOR USING OCR			
STUDENTS			
S. No	HT No	Name of the student	
1.	160721747007	Mohammad Rizwan	
2.	160721747032	Keshetty Ramya Sri	
3.	160721747058	Raparthi Bhoomika	

Course Outcomes: After the successful completion of this course, the student will be able to:

CO No	Course Outcome	BTL	PO
1PW868AD.1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program to the real-world problems.	Apply	PO1, PO2, PO3, PO6, PO8, PO11, PSO1, PSO2, PSO3
1PW868AD.2	Analyze and formulate the problem, and design solution of the selected project	Analyze	PO1, PO2, PO3, PO4, PO6, PO8, PO11, PSO1, PSO2, PSO3
1PW868AD.3	Evaluate different solutions based on economic and technical feasibility.	Evaluate	PO3, PO4, PO5, PO6, PO8, PO11, PSO1, PSO2, PSO3
1PW868AD.4	Effectively plan, develop and deploy a project and confidently perform all aspects of project management	Create	PO3, PO4, PO5, PO6, PO7, PO8, PO9, PO10, PO11, PSO1, PSO2, PSO3
1PW868AD.5	Demonstrate effective written and oral communication skills.	Create	PO2, PO5, PO7, PO8, PO9, PO10, PO11, PSO1, PSO2, PSO3

CO-PO-PSO MAPPING

3 – High, 2 – Moderate, 1 – Slight

PO / CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
1PW868AD.1	3	2	2			3		2	2	2	3	3	1	1
1PW868AD.2	3	3	2	3		2		2	2	2	3	3	3	3
1PW868AD.3		3	3		2	2		2		2	2	3	1	1
1PW868AD.4		3	3	3	3	2	2	3	3	3	3	3	3	3
1PW868AD.5		3			3		2	3	3	3	3	1	1	1



CO-PO-PSO MAPPING (With Performance Indicators)

PO / CO	PO 1	PO 2	PO3	PO4	PO5	PO 6	PO 7	PO 8	PO9	PO 10	PO 11	PS0 1	PSO 2	PSO3
1PW868AD .1	1.1.2 1.2.1 1.3.1 1.4.1	2.1.2 2.1.3 2.2.3 2.3.1 2.4.1	3.1.1 3.1.3 3.1.4 3.1.5 3.2.1 3.2.3 3.3.1 3.3.2			6.1.1 6.2.1 6.3.1 6.3.2 6.4.1		8.2.1 8.2.2 8.2.3	9.1.1 9.1.2 9.2.1 9.2.2 9.3.1 9.3.2	10.1.1 10.1.2 10.2.1	11.1.2 11.2.1 11.2.2 11.3.1 11.3.2	High	Slight	Slight
1PW868AD .2	1.1.2 1.2.1 1.3.1 1.4.1	2.1.1 2.1.2 2.1.3 2.2.3 2.3.1 2.4.1 2.4.3 2.4.4	3.1.1 3.1.3 3.1.4 3.1.5 3.1.6 3.2.1 3.2.3 3.3.1 3.3.2 3.4.1 3.4.2	4.1.1 4.1.2 4.1.3 4.2.1 4.3.1 4.3.2 4.3.3 4.3.4		6.1.1 6.3.1 6.3.2 6.4.2		8.1.1 8.1.2 8.2.1 8.2.2 8.2.3 8.3.1	9.1.1 9.1.2 9.1.3 9.2.1 9.2.2 9.3.1 9.3.2	10.1.1 10.1.2 10.2.1	11.1.2 11.2.1 11.2.2 11.3.2	High	High	High
1PW868AD .3		2.2.2 2.2.3 2.2.4 2.3.2 2.4.2 2.4.3 2.4.4	3.1.5 3.2.1 3.2.2 3.2.3 3.3.1 3.3.2 3.4.1 3.4.2	4.1.2 4.2.1 4.2.2 4.3.1 4.3.2 4.3.3 4.3.4	5.2.1 5.2.2 5.3.1 5.3.2	6.1.1 6.2.1 6.3.1 6.3.2 6.4.2		8.1.1 8.1.2 8.2.1 8.2.2 8.2.3 8.3.1		10.1.1 10.1.2 10.2.1	11.1.2 11.2.1 11.2.2 11.3.2	High	Slight	Slight
1PW868AD .4		2.3.1 2.3.2 2.4.1 2.4.2 2.4.3 2.4.4	3.1.6 3.2.1 3.2.2 3.3.2 3.4.1 3.4.2	4.1.2 4.1.3 4.2.1 4.2.2 4.3.1 4.3.3	5.1.1 5.1.2 5.2.1 5.2.2 5.3.1 5.3.2	6.1.1 6.2.1 6.3.1 6.3.2 6.4.2		8.1.1 8.1.2 8.2.1 8.2.2 8.2.3 8.3.1	9.1.1 9.1.2 9.1.3 9.2.1 9.2.2 9.3.1 9.3.2	10.1.1 10.1.2 10.2.1 10.3.1 10.3.2	11.1.1 11.1.2 11.2.1 11.2.2 11.3.2	High	High	High
1PW868AD .5					5.1.1 5.1.2 5.2.1 5.2.2 5.3.1 5.3.2			8.1.1 8.1.2 8.2.1 8.2.2 8.2.3 8.3.1	9.1.1 9.1.2 9.1.3 9.2.1 9.2.2 9.3.1 9.3.2	10.1.1 10.1.2 10.2.1 10.3.1 10.3.2	11.1.1 11.1.2 11.2.1 11.2.2 11.3.2	Slight	Slight	Slight

Project Specific Learning Outcomes

LO#	Outcome Description
1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program to address real-world multilingual communication challenges through OCR and translation technologies.
2	Analyze and formulate the problem of multilingual text recognition from images, and design an effective solution using OCR and language translation techniques.
3	Evaluate different OCR tools and translation services based on their technical performance and economic feasibility for real-time multilingual signboard translation.
4	Effectively plan, develop, and deploy the multilingual signboard translator project, and confidently perform all aspects of project management including task allocation, timeline tracking, and integration.
5	Demonstrate effective written and oral communication skills through comprehensive project documentation, presentations, and user demonstrations of the multilingual signboard translator.

PO Mapping:

Level of Mapping - 3: High; 2: Moderate; 1-Low

PO#	PO Description	Mapped Wks	PO#		Mapped Wks
PO1	Engineering Knowledge	WK1, WK2, WK3, WK4	PO7	Ethics	WK9
PO2	Problem Analysis	WK1, WK2, WK3, WK4	PO8	Individual & Collaborative Team Work	
PO3	Design / Development of Solutions	WK5	PO9	Communication	
PO4	Conduct Investigations of complex problems	WK8	PO10	Project Management & Finance	
PO5	Engineering Tool usage	WK2, WK6	PO11	Life-long Learning	WK8
PO6	The Engineer & The World	WK1, WK5, WK7			

SDG Mapping:

SDG	Mapped Indicator	SDG	Mapped Indicator	SDG	Mapped Indicator
1 NO POVERTY		7 AFFORDABLE AND CLEAN ENERGY		13 CLIMATE ACTION	
2 ZERO HUNGER		8 DECENT WORK AND ECONOMIC GROWTH		14 LIFE BELOW WATER	
3 GOOD HEALTH AND WELL-BEING		9 INDUSTRY, INNOVATION AND INFRASTRUCTURE	9.5.1, 9.5.2, 9.c.1	15 LIFE ON LAND	
4 QUALITY EDUCATION	4.4.1, 4.7.1	10 REDUCED INEQUALITIES	10.2.1, 10.3.1	16 PEACE, JUSTICE AND STRONG INSTITUTIONS	
5 GENDER EQUALITY		11 SUSTAINABLE CITIES AND COMMUNITIES	11.2.1, 11.7.1, 11.7.2	17 PARTNERSHIPS FOR THE GOALS	
6 CLEAN WATER AND SANITATION		12 RESPONSIBLE CONSUMPTION AND PRODUCTION			

QUALITY ANALYSIS

Project collaboration (Tick ✓ Whichever is applicable and present the details below):

Industry	Research Lab	Professional Society	Community ✓	Alumni	Regulating agency	Govt.	Other
Details: Designed to aid multilingual public access for travelers, migrants, and can be prospectively used for the Ministry of Tourism, Government of India.							

Project Achievements:

Does the Abstract (prepared/ reviewed / Approved) have the following stated outcomes mentioned?	Yes
---	-----

S. No	Description	Details
1.	Prototype development	--
2.	Technology-based solution developed	Multilingual Signboard Translator Using OCR
3.	Publication	In Progress
4.	Start-up incubation	--
5.	Competition prizes	--
6.	Community / Industry / HEI service	Designed to aid multilingual public access for travelers and migrants
7.	SDG Indicators	4.4.1, 4.7.1, 9.5.1,9.5.2, 9.c.1,10.2.1, 10.3.1,11.2.1, 11.7.1, 11.7.2
8.	Grants / Financial Aid	--
9.	Multi-disciplinary	--
10.	Internship / Employment	--
11.	Self-employment skill	PROSPECTIVE: Application can be made available for the Ministry of Tourism and Travel Apps
12.	Use of real-time data	Images captured from the Signboards across India

Signature of the Guide

Signature of the Project Coordinator

Signature of the Assessment Coordinator

Signature of the Head of the Department

RESOURCES

